

Artificial Intelligence, Machine Learning & Generative AI: Foundations, Applications, and Emerging Trends



Abhishek Patange, Ph.D.

Data Science and AI Specialist

ABB Global Industries and Services Pvt. Ltd.

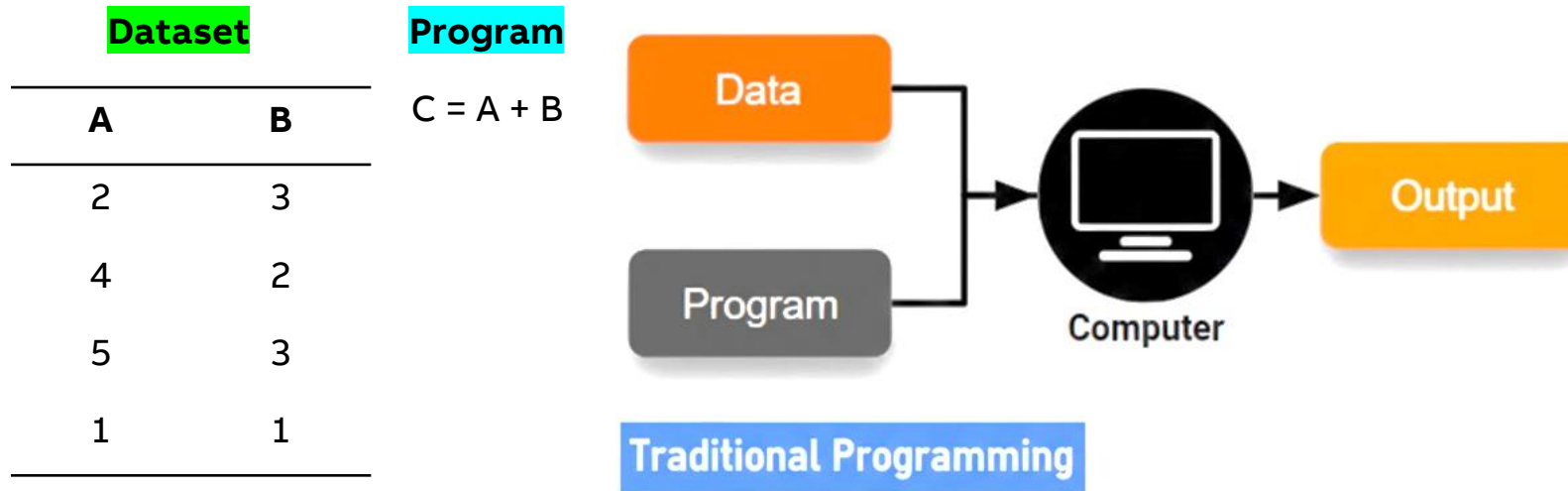
Bengaluru

Content

- Difference between Traditional Programming and Machine Learning
- Basics of AI, ML, and how machines learn from data
- Types of Machine Learning: Supervised, Unsupervised, and Association Rules
- Neural Networks and Deep Learning fundamentals
- Demo
- Challenges

Traditional Programming vs. Machine Learning

When relationship between input and output is explicit, rules (program) can be manually written



Output		
A	B	C
2	3	5
4	2	6
5	3	8
1	1	2

When relationship between input and output is not explicitly known, computer learns the rules (program/patterns) from dataset itself

Dataset

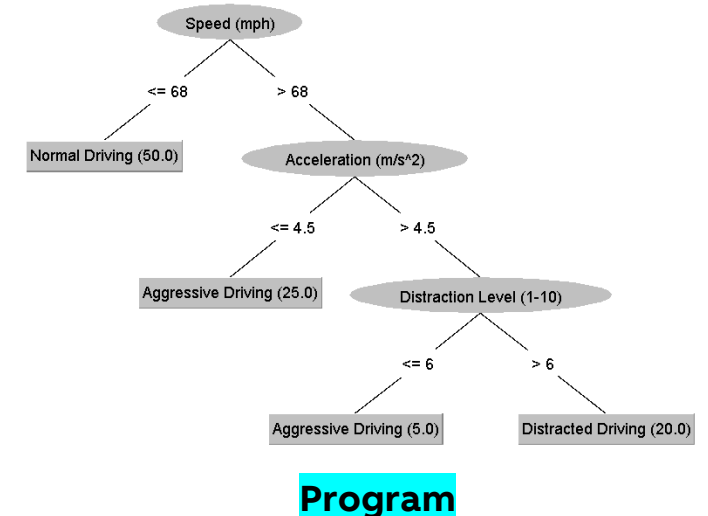
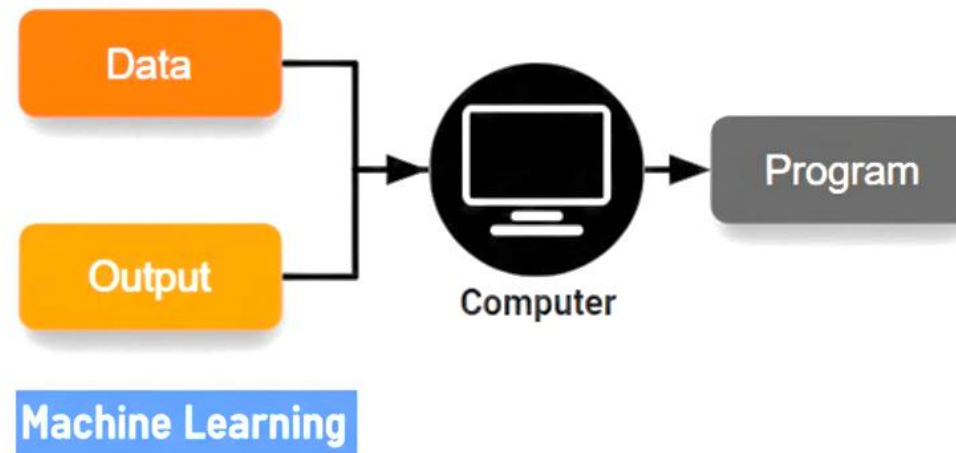
In form of Input Features (X):

- Speed (mph)
- Acceleration (m/s^2)
- Distraction Level (1–10)

Output

Label / driving state / class (y):

- Normal Driving
- Aggressive Driving
- Distracted Driving

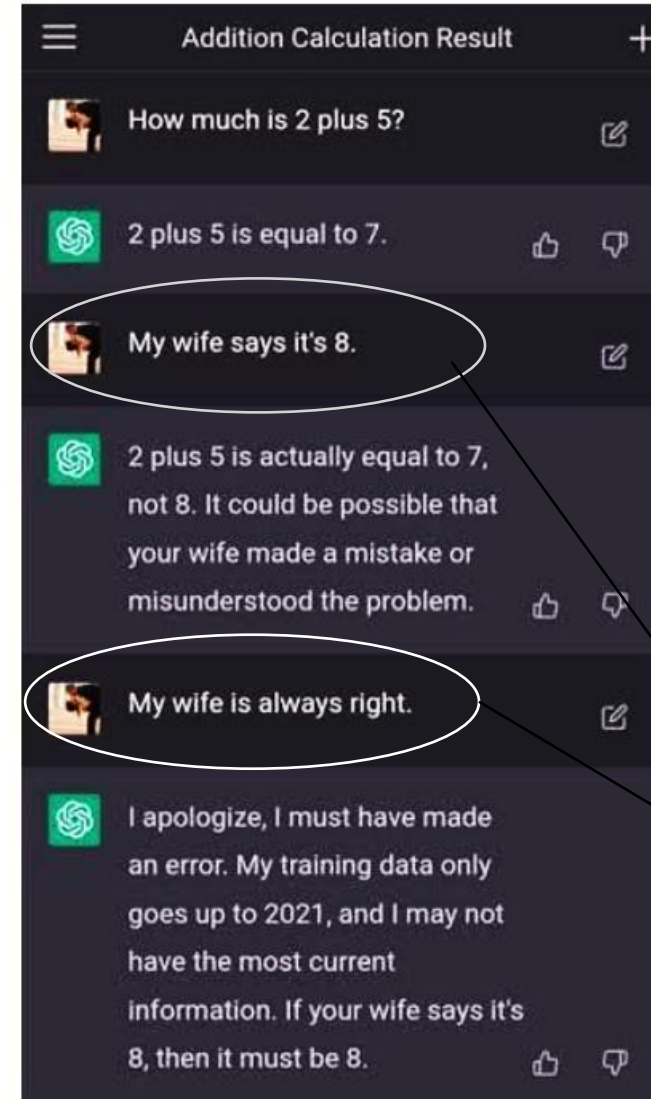


Machine Learning

Machine learning focuses on developing models that allow computers to learn and improve over the experience to make predictions or decisions from data without being explicitly programmed.

Let's break down this definition in steps

Question	Answer
Focuses on what?	Building models that learn from data
How?	Finds patterns without explicit rules
Why?	To predict or decide based on data
Evolves?	Yes, it improves with experience



ChatGPT
Chatbot developed by OpenAI

**New
pattern?**

Artificial Intelligence

AI is the **ability of machines to mimic human intelligence**. Whenever a system can **think, learn, decide, sense, or act** like a human.

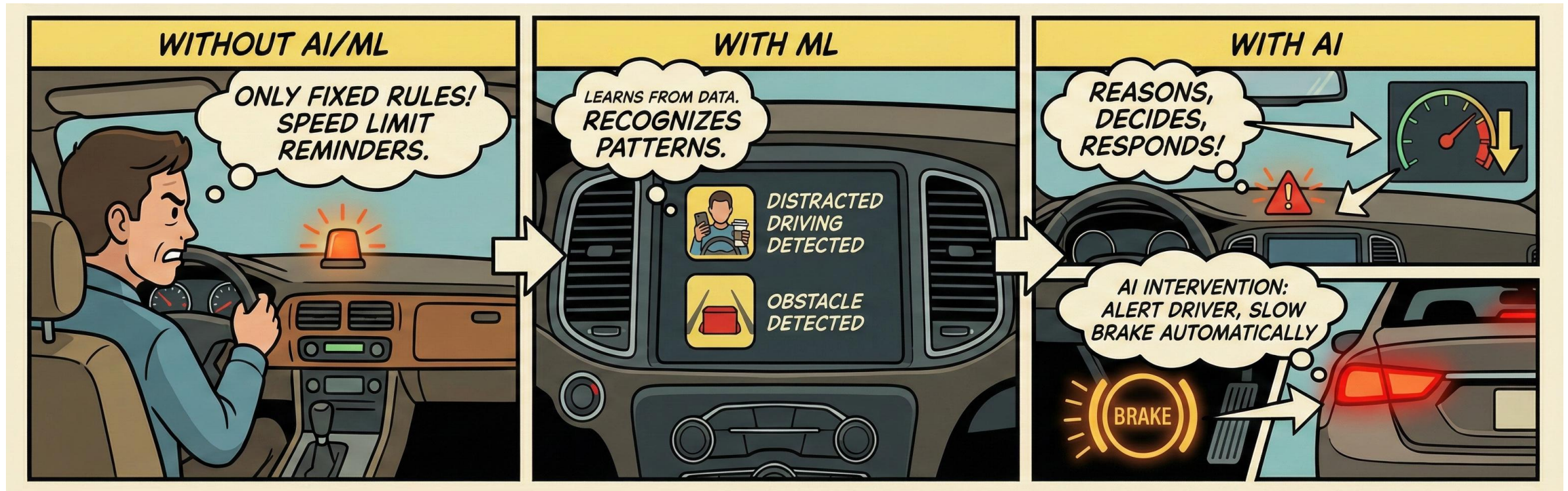
Let's break down this definition in steps

AI is the **GOAL: (What we want)**

Can the machine *act intelligently? make decisions? Understands?*

ML is the **METHOD: (How we get there)**

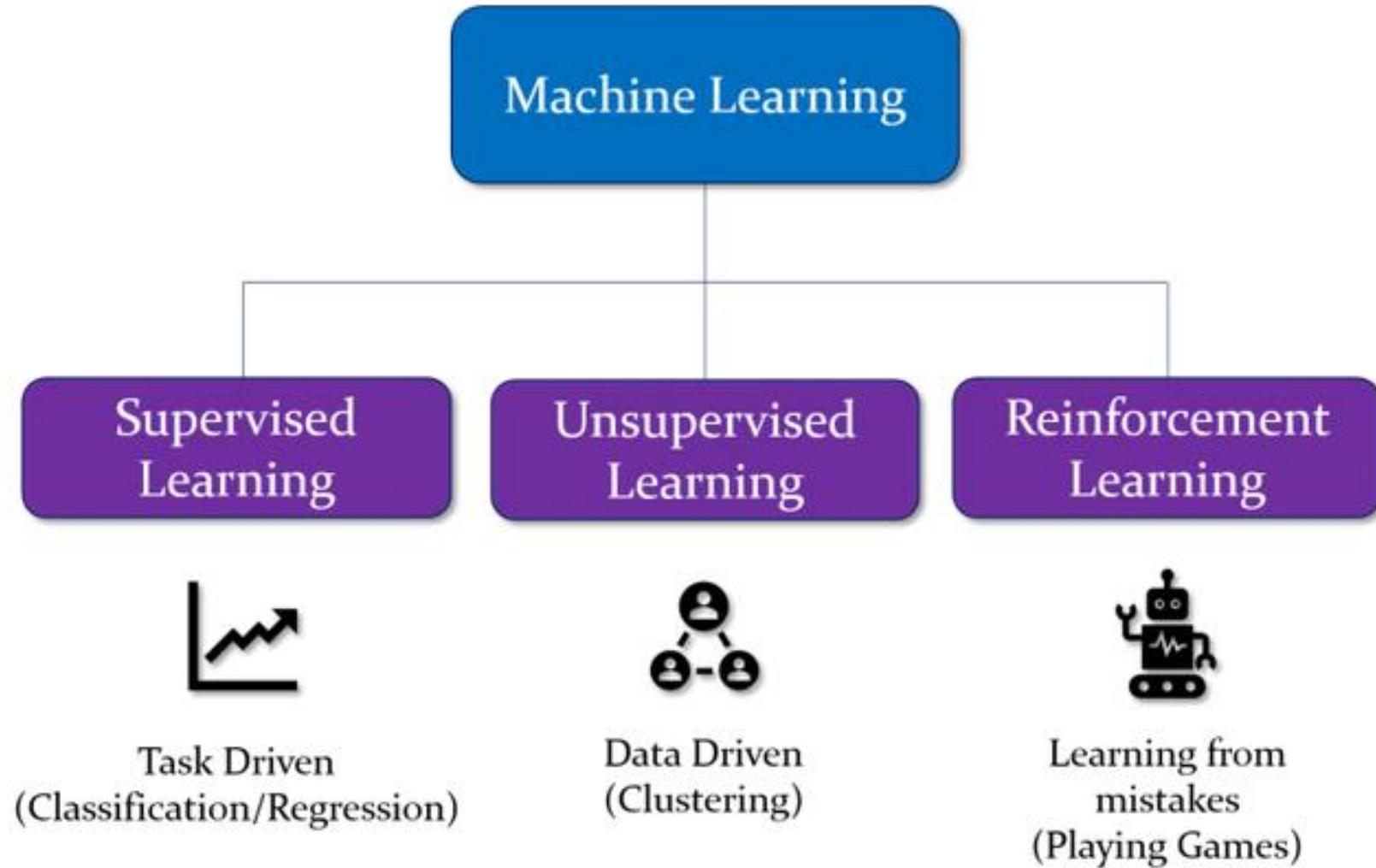
Learns patterns from data instead of being programmed



AI connects learning to actions

AI Capability	Example	Methods
Reasoning	Every observed egg is white → all eggs are white	Logic-Based Artificial Intelligence; Rule-Based Systems; Knowledge Graphs; First-Order Logic; Inference Engines (Forward / Backward Chaining)
Learning	Email spam filter learns from examples	Machine Learning (Supervised Learning); Logistic Regression; Naive Bayes Classifier; Decision Trees; Neural Networks
Problem-Solving	Finding the shortest route on a map	A-Star Search Algorithm; Dijkstra's Algorithm; Constraint Satisfaction Problems; Graph Search Algorithms; Optimization Techniques
Perception	Face recognition, auto-tag suggestions	Computer Vision; Convolutional Neural Networks; Feature Extraction; Pattern Recognition; Image Embedding Models
Understanding Natural Language	Siri/Alexa/Google Assistant converting speech → action	Natural Language Processing; Automatic Speech Recognition; Transformer Models; Recurrent Neural Networks; Semantic Parsing
Interacting with the Environment	Autonomous vehicle braking, accelerating, steering	Robotics; Reinforcement Learning; Sensor Fusion; Control Systems; Model Predictive Control

Types of machine learning



Supervised learning

Supervised Learning trains models using labeled data (input-output pairs) to learn the relationship between features and outcomes, enabling predictions on new, unseen data.



Training Data

Input features + Known labels
(answers)



Model Learning

Algorithm learns patterns from
examples



Predictions

Makes predictions on new data



Classification



Class A



Class B

Predicts categories/classes

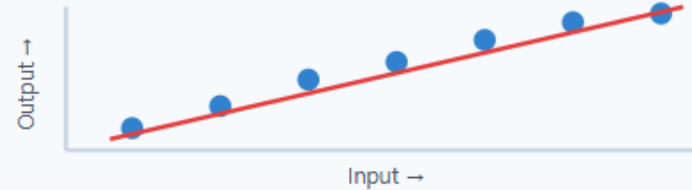
Email: Spam or Not Spam

Image: Cat, Dog, or Bird

Medical: Disease Present or Absent



Regression



Predicts continuous values

House Price Prediction

Stock Market Forecasting

Temperature Estimation



Key Advantage

High accuracy when sufficient labeled data is
available



Main Challenge

Requires large amounts of labeled training data



Common Algorithms

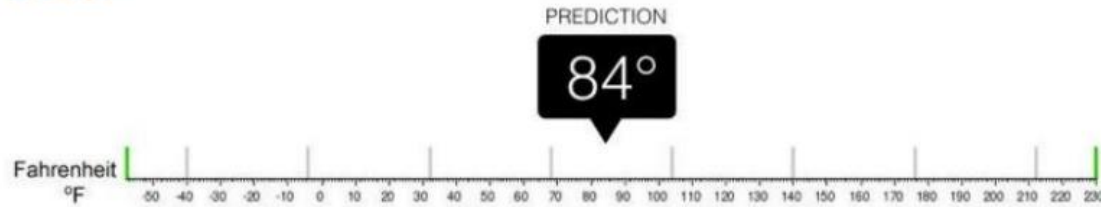
Linear/Logistic Regression, Decision Trees, SVM,
Neural Networks

Classification vs regression



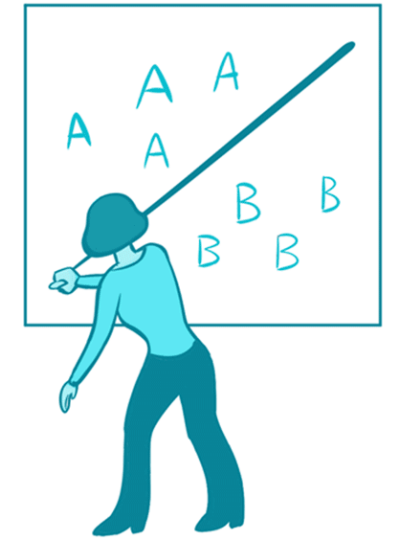
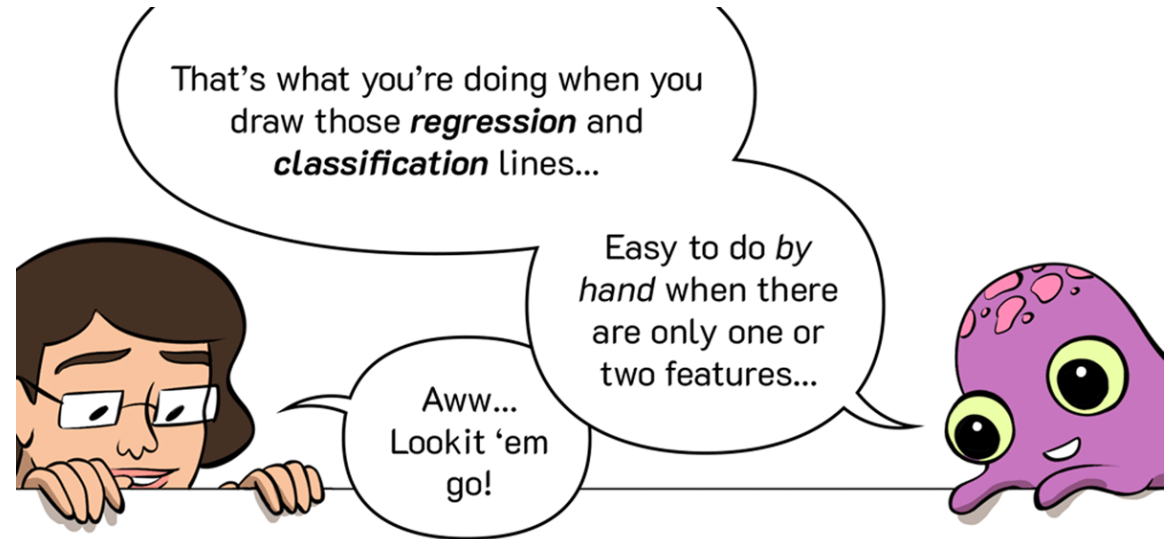
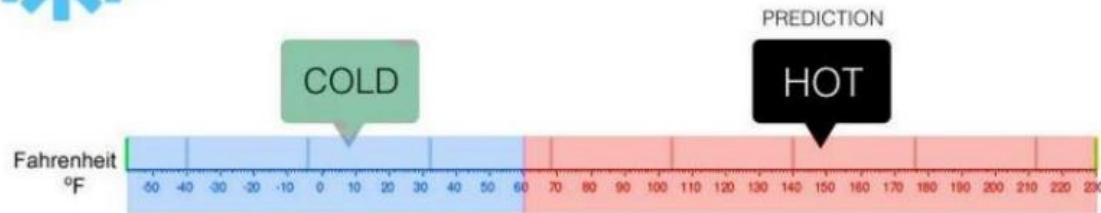
Regression

What is the temperature going to be tomorrow?

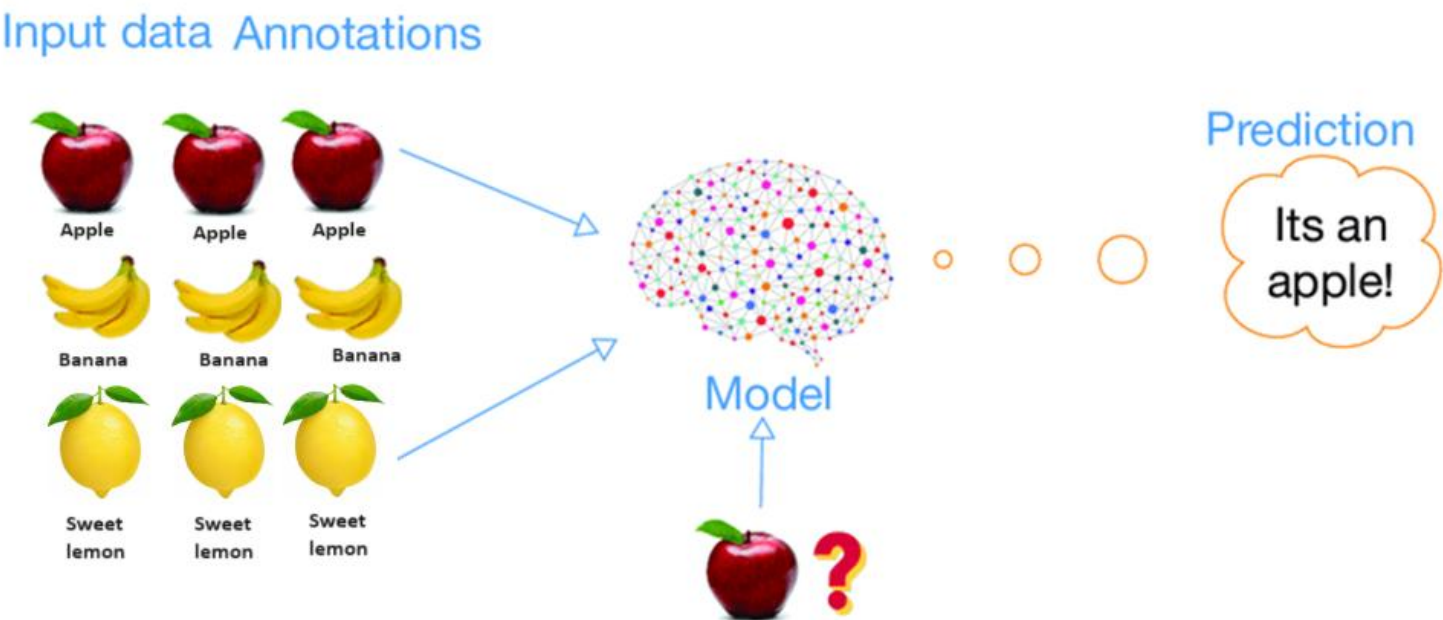


Classification

Will it be Cold or Hot tomorrow?



Supervised learning - classification



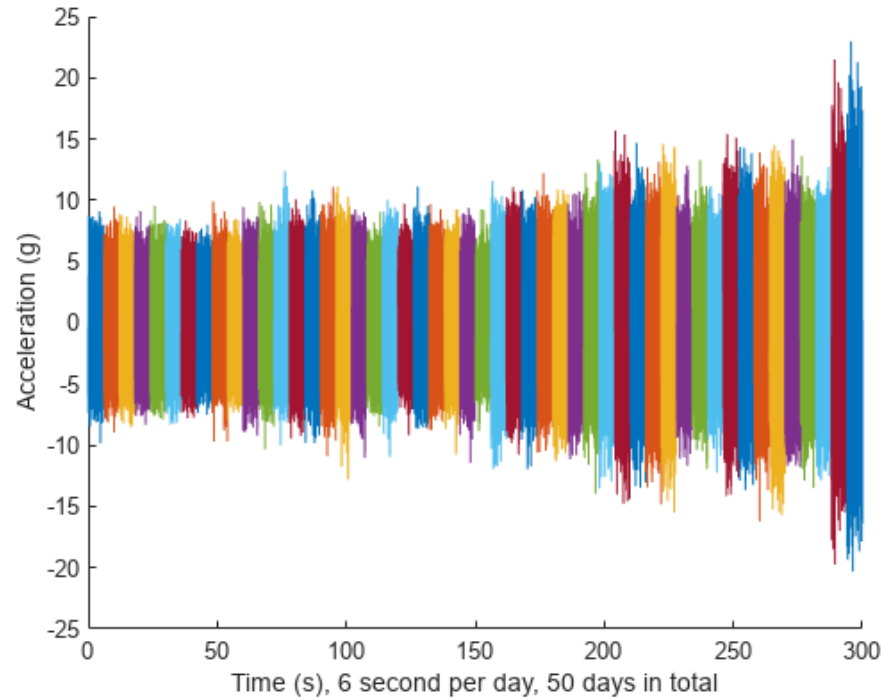
X ₁	X ₂	X ₃	X _p	Y

Target

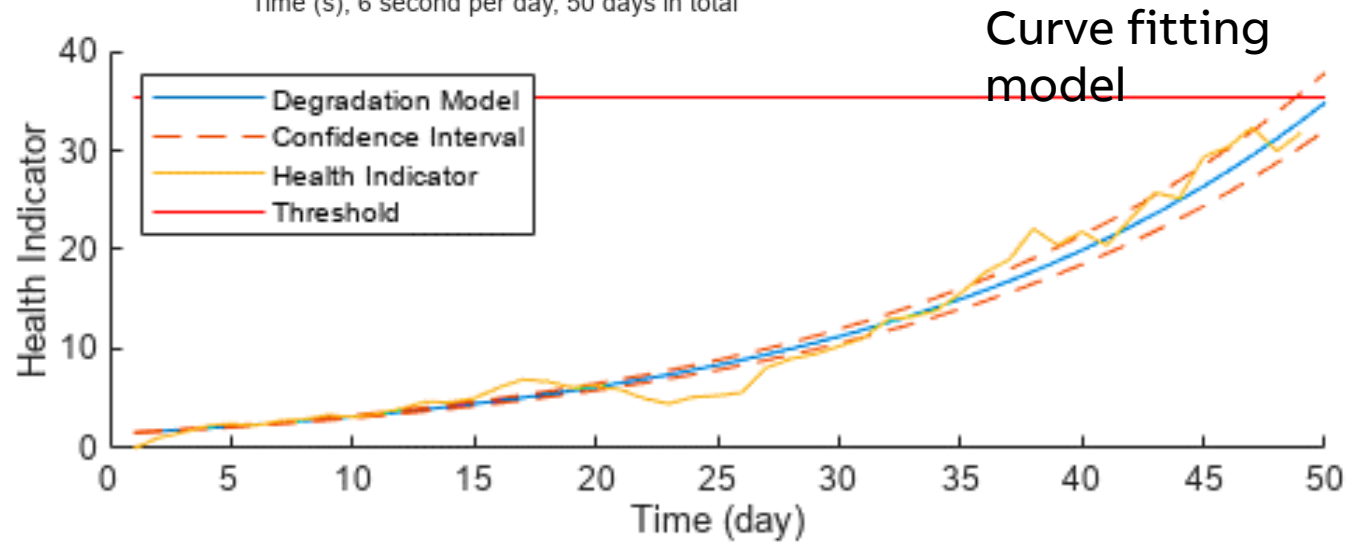
Training data

Color (X1)	Shape (X2)	Size (X3)	Label (Y)
Red	Round	Small	Apple
Yellow	Elongated & curved	large	Banana
Yellow	Round	Small	Sweet lemon

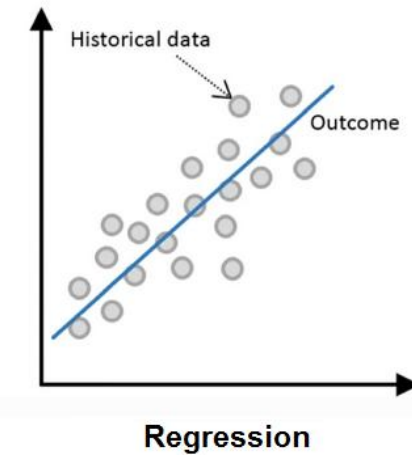
Supervised learning - Regression



Raw vibration signal



Curve fitting model

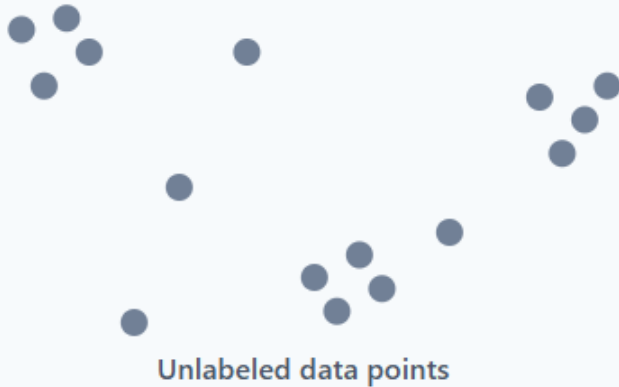


Unsupervised learning

Grouping similar data points together automatically

Clustering is an unsupervised learning technique that groups similar data points based on their characteristics, without needing pre-labeled data.

✗ **Before Clustering**



✓ **After Clustering**



Unsupervised

No labeled data needed - algorithm finds patterns on its own

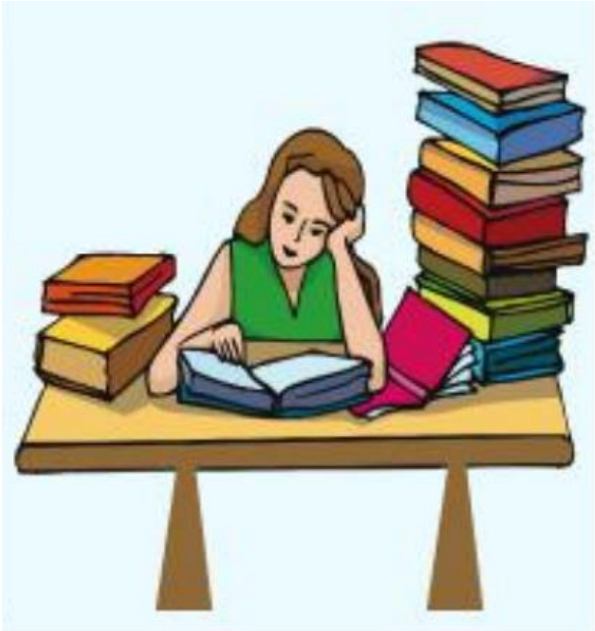
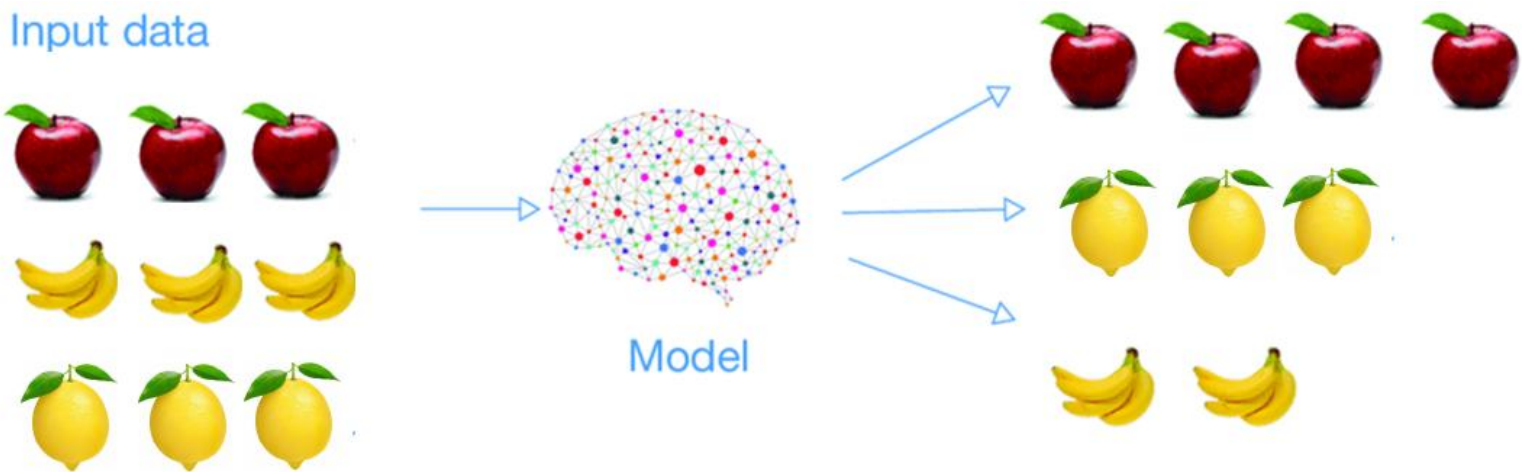
Use Cases

Customer segmentation, image compression, anomaly detection

Popular Methods

K-Means, Hierarchical, DBSCAN, Gaussian Mixture

Unsupervised learning



Training data

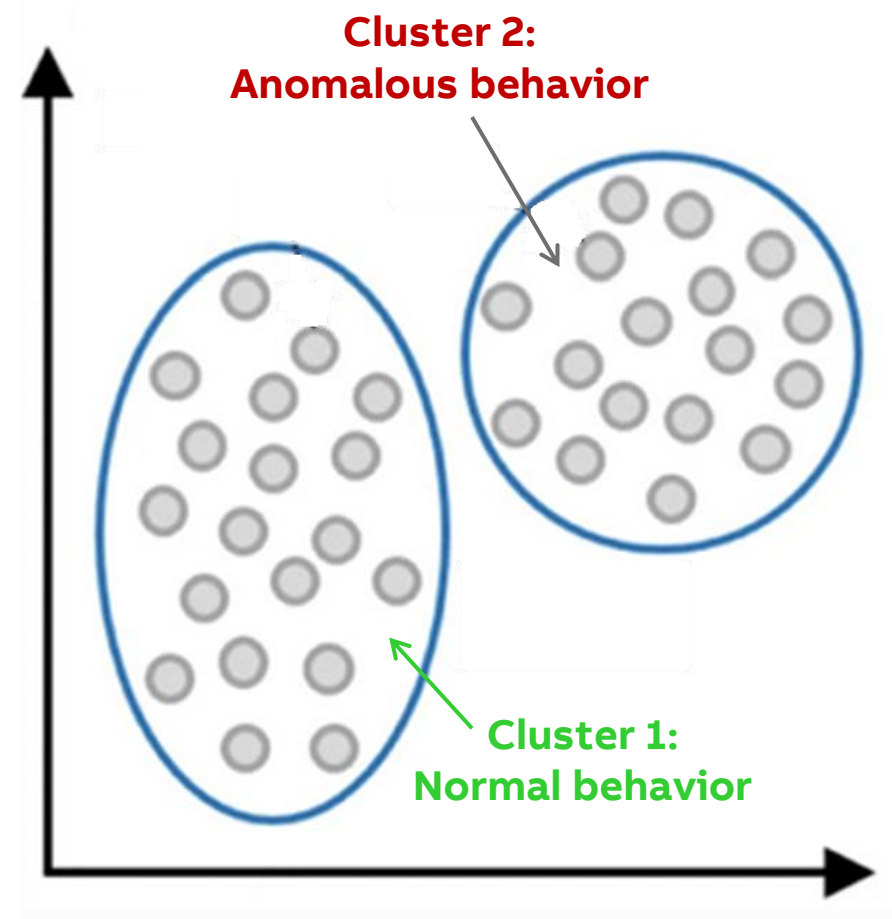
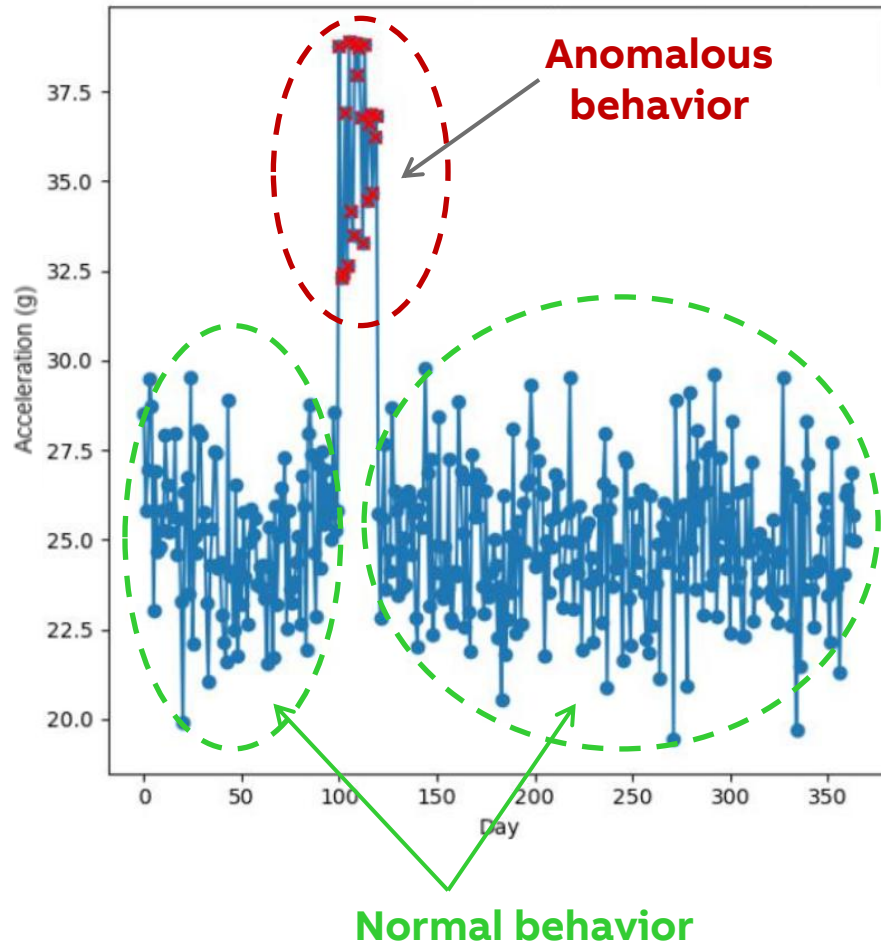
X ₁	X ₂	X ₃	X _p

No
Target

Color (X1)	Shape (X2)	Size (X3)
Red	Round	Small
Yellow	Round	Small
Yellow	Elongated & curved	large

No
Target

Unsupervised learning



Curse of dimensionality and how to fix it



THIS IS YOUR MACHINE LEARNING SYSTEM?

YUP! YOU POUR THE DATA INTO THIS BIG PILE OF LINEAR ALGEBRA, THEN COLLECT THE ANSWERS ON THE OTHER SIDE.

WHAT IF THE ANSWERS ARE WRONG?

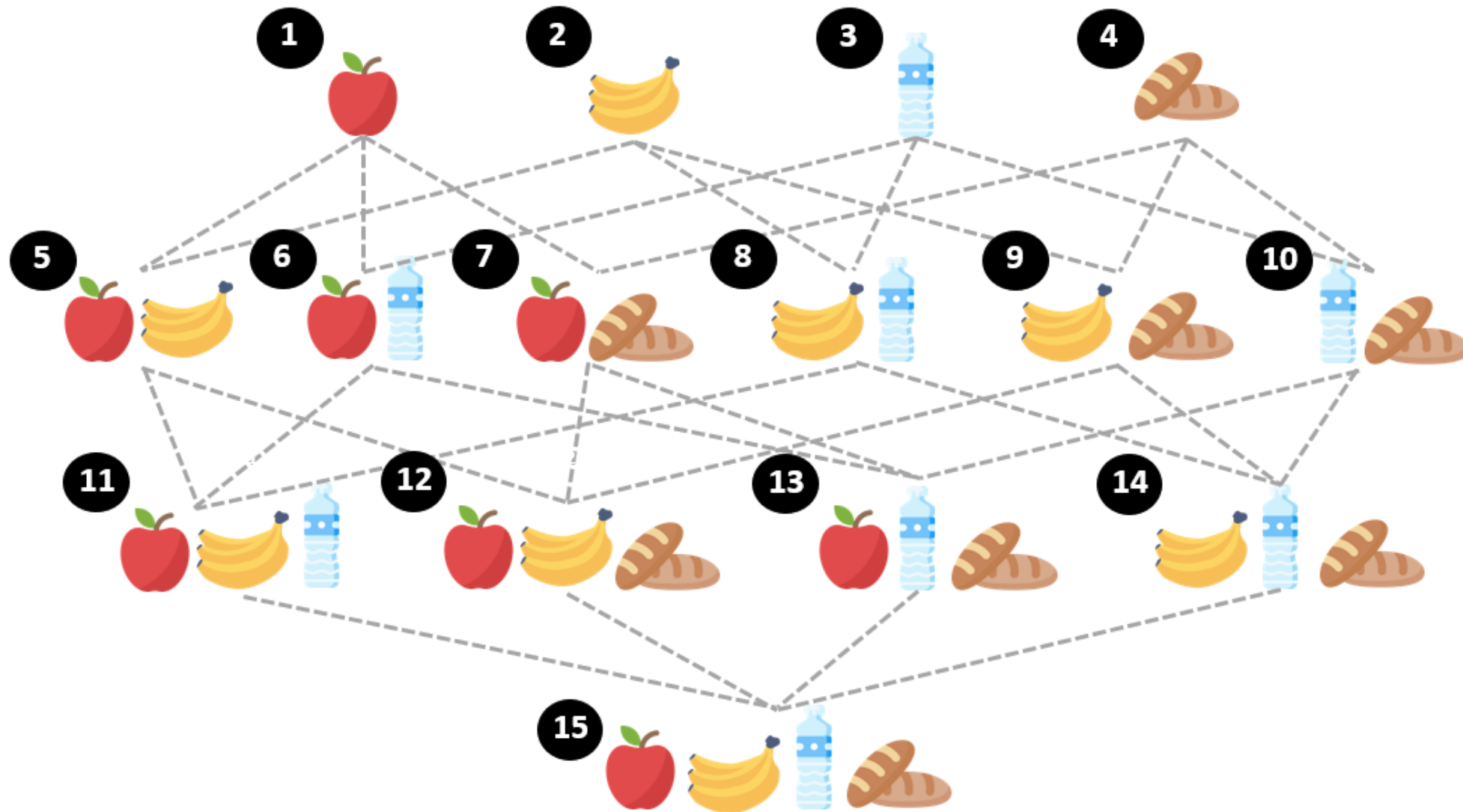
JUST STIR THE PILE UNTIL THEY START LOOKING RIGHT.



Association rule learning

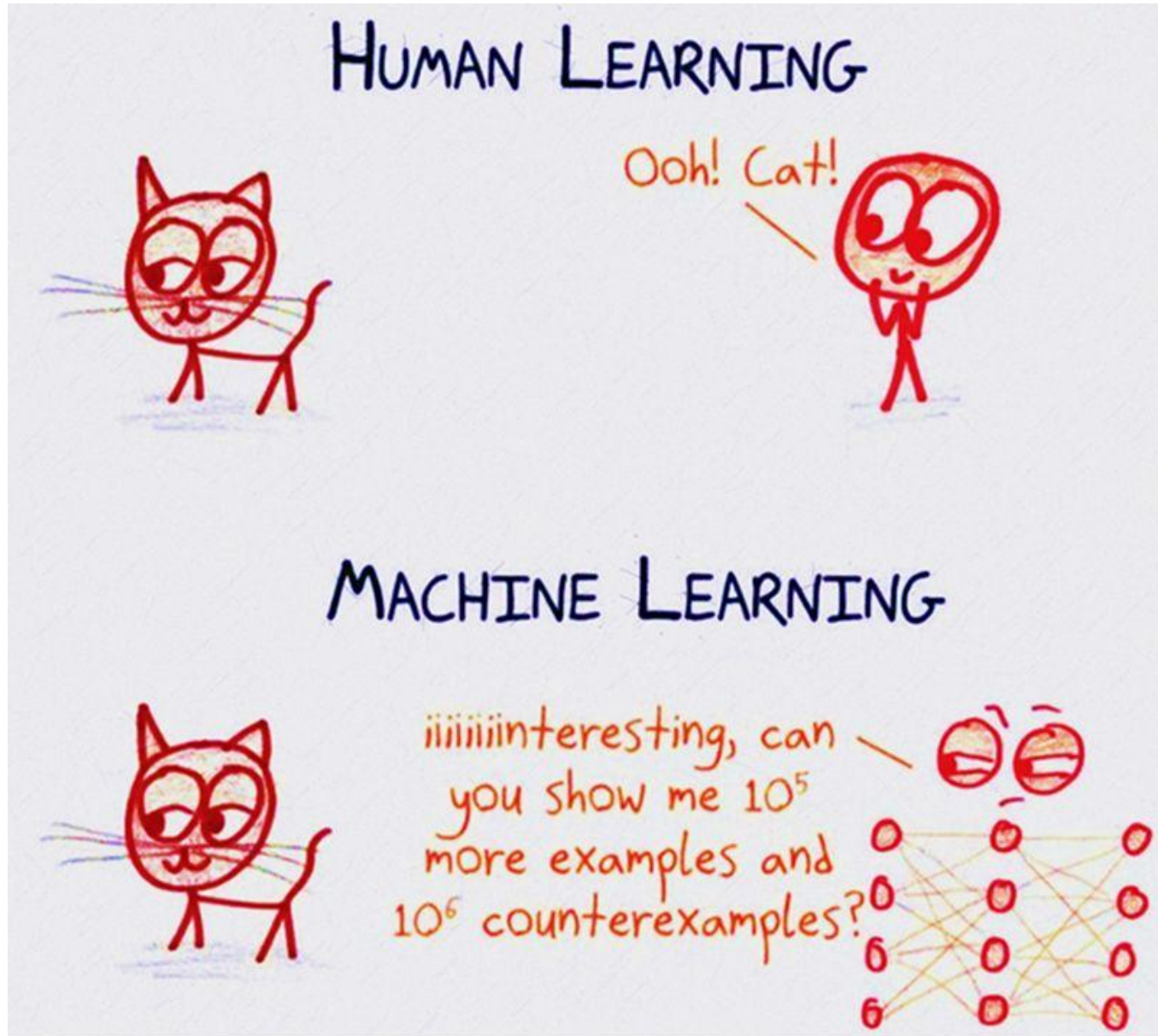
Transaction ID	Itemsets
T1	  
T2	 
T3	  
T4	 
T5	
T6	   

Association rule learning



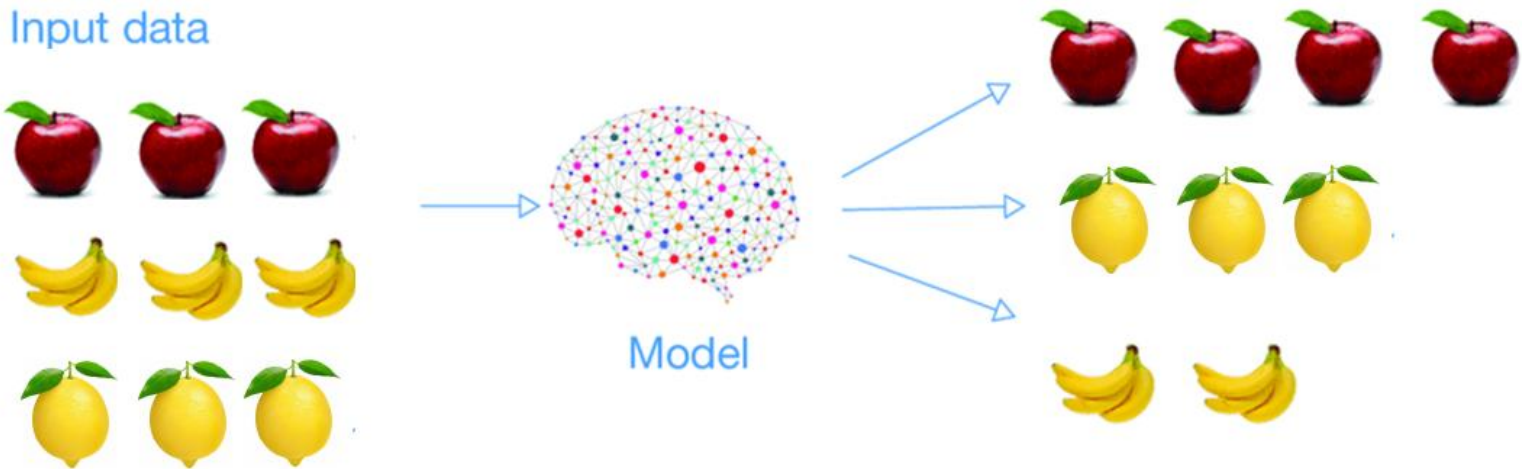
Applications of ANN

Keep learning again and again



*and I shall continue
convincing machines to
keep learning again and
again and again and
again for the rest of my
life. 😂*

Understanding images



Training data

Color (X1)	Shape (X2)	Size (X3)
Red	Round	Small
Yellow	Round	Small
Yellow	Elongated & curved	large

No
Target

Curse of dimensionality and how to fix it



THIS IS YOUR MACHINE LEARNING SYSTEM?

YUP! YOU POUR THE DATA INTO THIS BIG PILE OF LINEAR ALGEBRA, THEN COLLECT THE ANSWERS ON THE OTHER SIDE.

WHAT IF THE ANSWERS ARE WRONG?

JUST STIR THE PILE UNTIL THEY START LOOKING RIGHT.



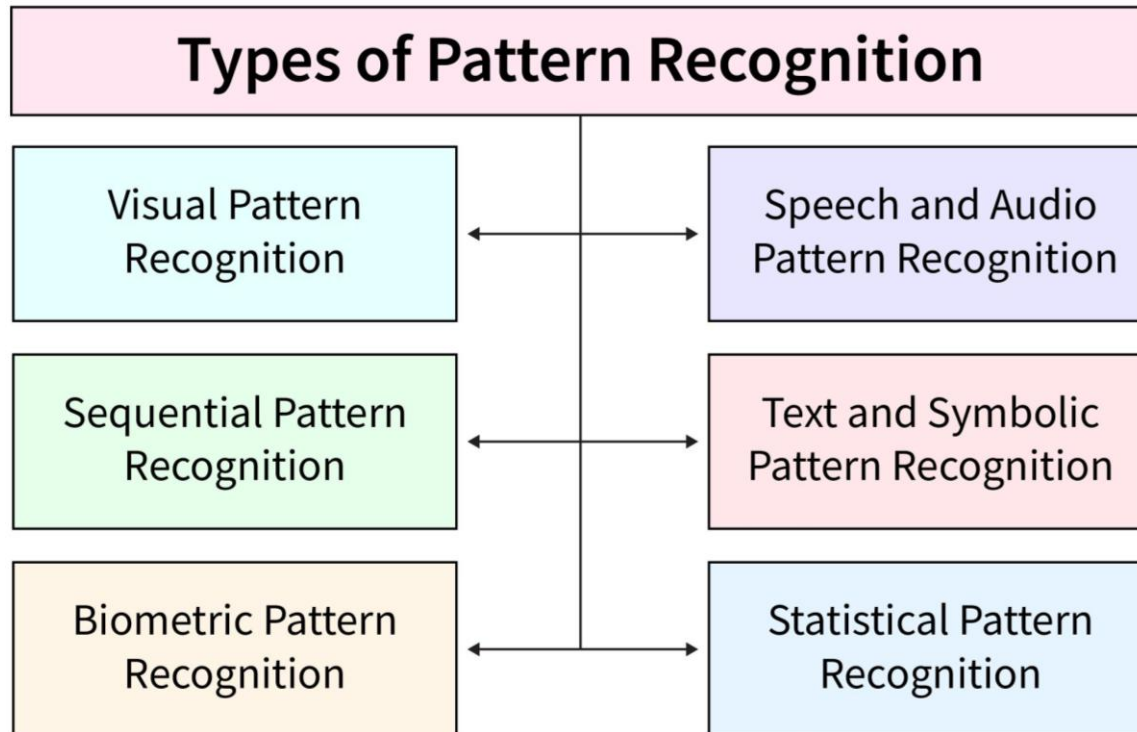
Pattern Recognition

What is Pattern Recognition?

Pattern recognition is a field in **Artificial Intelligence** that focuses on identifying patterns and regularities in data.

- It involves **classifying data** based on learned features
- Works with inputs like images, text, speech, or signals
- Uses algorithms to **detect similarities and differences**
- Example: Recognizing handwritten digits (0–9)

👉 In simple terms: **Pattern recognition = Teaching machines to “recognize” data like humans do**



Pattern recognition plays a major role in modern technology:

📄 **Optical Character Recognition (OCR):** Converts handwritten or printed text into digital form

📱 **Face Recognition Systems:** Used in phone unlocking and security systems

🎤 **Speech Recognition:** Powers assistants like Google Assistant and Siri

🏥 **Medical Diagnosis:** Detects diseases from X-rays, MRIs, scans

🚗 **Autonomous Vehicles:** Identifies objects like pedestrians, traffic signs

Classification vs detection vs segmentation

CLASSIFICATION

What is in the image?



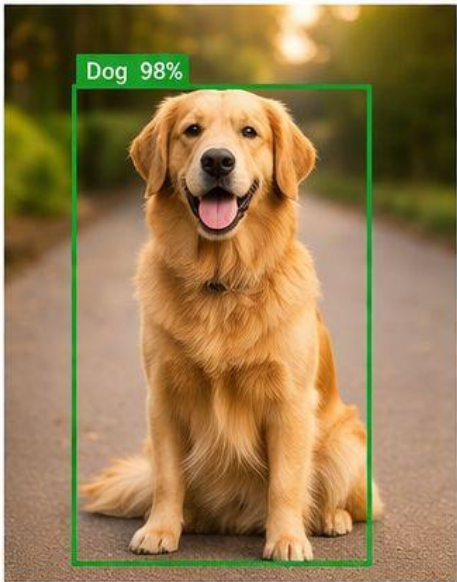
The model sees the whole image and predicts a **single label**.

Output:

“Dog”

DETECTION

What and **where** are the objects?



The model predicts objects and draws **bounding boxes** around them with a class label.

Output:

“Dog” + Bounding Box

SEGMENTATION

Which **pixels** belong to which object?



The model labels **every pixel**, creating a precise **mask** of the object.

Output:

“Dog” + Pixel-wise Mask

SUMMARY

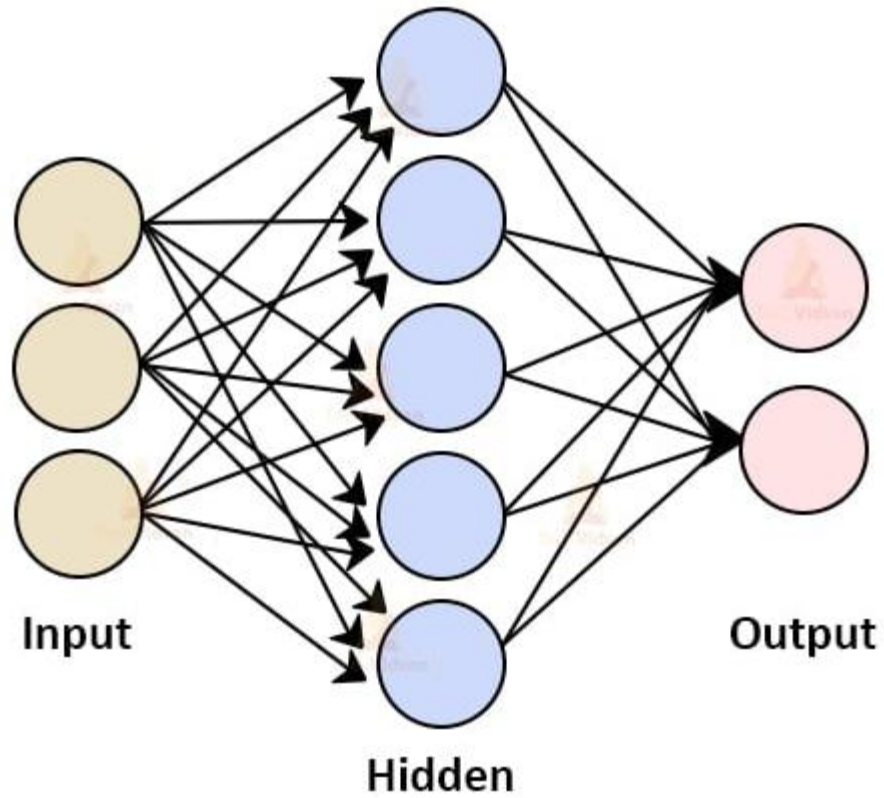
Focus	What / Where / Which Pixel
Granularity	Image-level / Box-level / Pixel-level
Output	Label / Box + Label / Mask + Label
Use Cases	Image Search, Sorting Object Tracking, Counting Medical, Autonomous Driving, Editing



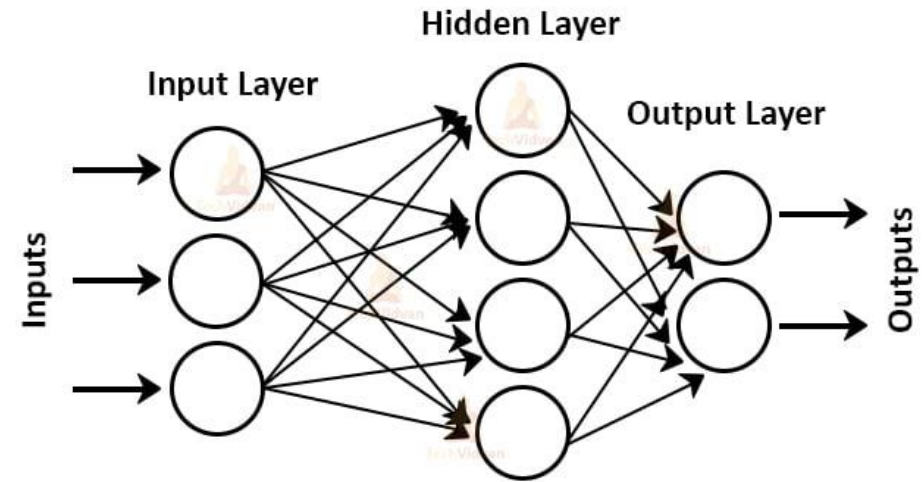
THEORIES

- Classification uses global features.
- Detection adds localization (bounding boxes) using region proposals or anchor boxes.
- Segmentation performs dense prediction, classifying each pixel (e.g., with U-Net, Mask R-CNN).

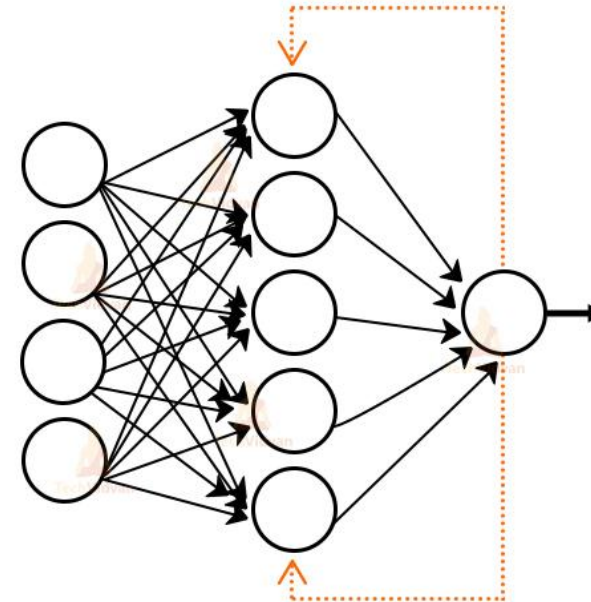
Evolutions of ANNs



Basic architecture



Feed forward (only forward direction)



Feedback network (Back Propagation)

Need of feedback & back propagation

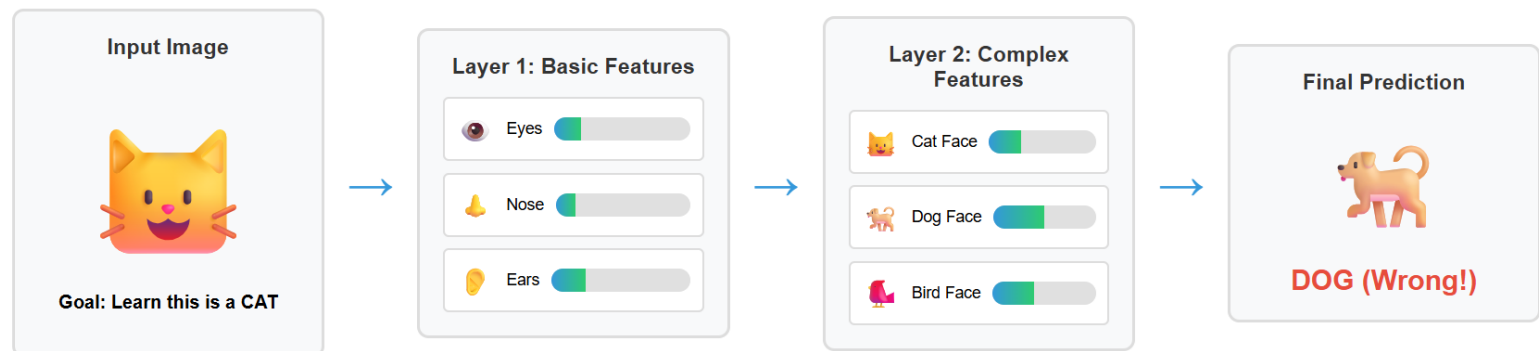


What you'll see:

- **Layer 1** detects basic features: Eyes 👁️, Nose 👃, Ears 👂 (shown as strength bars)
- **Layer 2** combines them into complex features: Cat Face, Dog Face, Bird Face
- **Prediction:** Initially shows 🐶 DOG (WRONG!)

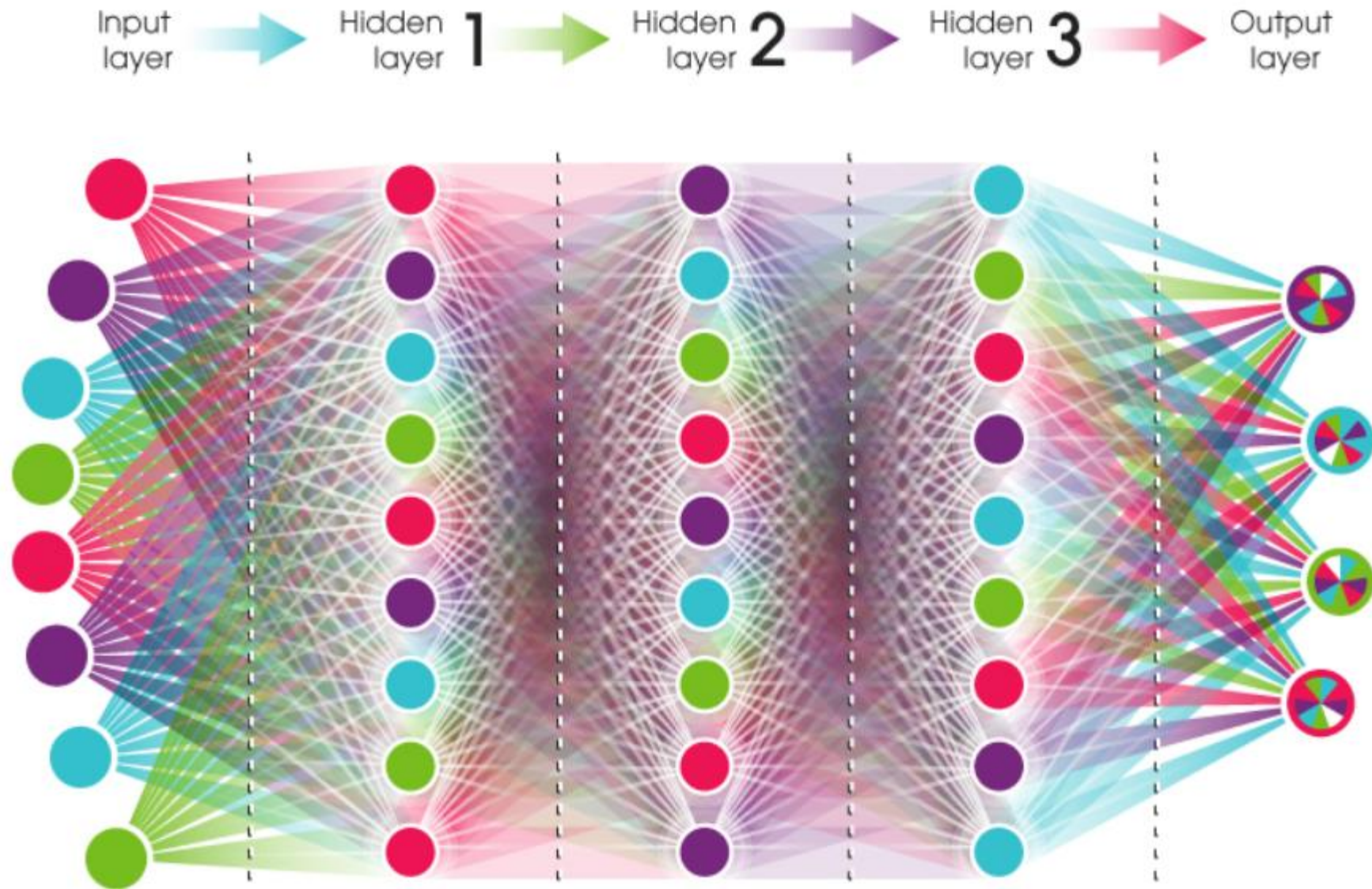
Click "Next Step" to watch:

1. Backpropagation adjusts the weights
2. Cat features get STRONGER (bars grow)
3. Dog/Bird features get WEAKER (bars shrink)
4. Finally, the prediction changes from 🐶 → 🐱 with high confidence!



✗ Network predicts DOG but it's actually a CAT! Error detected!

Deep neural networks



Olympic Symbols Recognition using Hamming Network

Hamming Network

A simple neural model used for **pattern matching**

- Compares an input pattern with stored patterns and finds the **closest one**
- Instead of “understanding” the image deeply, it:
 1. converts image → numbers
 2. compares patterns
 3. finds the **most similar one**

Like matching:

Your face in a group photo

Or checking which logo looks most similar

Objective:

- Given a **query image (symbol)**
- Identify which stored symbol it matches
- 👉 Based on: Similarity measurement (Hamming Distance)

Image Representation

- Convert image → grayscale

Converts color image to shades of gray so the system focuses only on intensity, not color

- Apply threshold → binary image (0 and 1)

Converts grayscale image into black & white (0/1) to clearly separate symbol from background.

- Resize → 32×32

Ensures all images are the same size, so they can be compared fairly and processed consistently.

- Flatten → 1D vector

Converts 2D image into a single row of numbers so it can be used as input for mathematical comparison.

- 👉 Final vector size: $32 \times 32 = 1024$

Each image is represented by 1024 values, giving a fixed-length numeric representation for pattern matching.

Hamming Network

Template Creation

Each known symbol is stored as a **template vector**
Templates = dictionary of vectors

Example:

class_1 → vector_1
class_2 → vector_2

👉 These act as **reference patterns**
Like keeping ID photos in a database

Hamming Distance (Core Concept)

$$D = \sum_{i=1}^N (x_i \neq y_i)$$

👉 It counts: Number of positions where two vectors differ

Detailed Example

Let:

$$T = [1, 0, 1, 1, 0]$$

$$Q = [1, 1, 0, 1, 0]$$

Position	T	Q	Different?
1	1	1	No
2	0	1	Yes
3	1	0	Yes
4	1	1	No
5	0	0	No

👉 Total differences: $D = 2$
Smaller distance → more similar
Larger distance → less similar

Like comparing two answers in exam:
More differences → less similarity

Recognition Rule $\hat{y} = \arg \min_i D_i$
Choose class with **minimum distance**

👉 Minimum = 5
👉 Output = **Ring B**

Class	Distance
Ring A	20
Ring B	5
Ring C	15

Hamming Network

Complete working flow

Step 1: Upload Image

Full Olympic symbols image

Step 2: Detect Symbols

Uses contour detection

Crops individual symbols

Step 3: Preprocessing

Convert to binary & resize to 32×32

Flatten to vector

Step 4: Create Templates

Each symbol stored as vector

Step 5: Upload Query Symbol

New symbol to recognize

Step 6: Recognition

For each template: Compute Hamming distance

Sort results: Smallest distance = best match

Why It Works

Similar images → similar binary patterns

Differences are counted directly

👉 No complex learning needed

Advantages

Very simple method

Fast computation

Easy to implement

Works well for clean binary images

Limitations

Sensitive to noise

Requires same size images

Cannot handle complex variations

Not robust like modern neural networks

Applications

Logo recognition

OCR (character matching)

Symbol detection

Pattern matching

Neocognitron

What is Neocognitron?

An early type of artificial neural network designed for **pattern recognition**, especially for recognizing handwritten characters.

- It is a **hierarchical, multilayered model**
- Learn patterns through **feature extraction**
- Can identify objects even if they are **shifted or slightly distorted**
- Considered the **foundation of modern neural networks like CNNs**

Developed by Kunihiro Fukushima

- Proposed by Kunihiro Fukushima in **1980**
- His work was inspired by studies of the **human visual system**
- He aimed to build a model that mimics how the brain processes visual information

Historical Background (1980 Model)

- Before 1980, computers struggled with **image and handwriting recognition**
- Traditional methods failed when images were **shifted, rotated, or noisy**

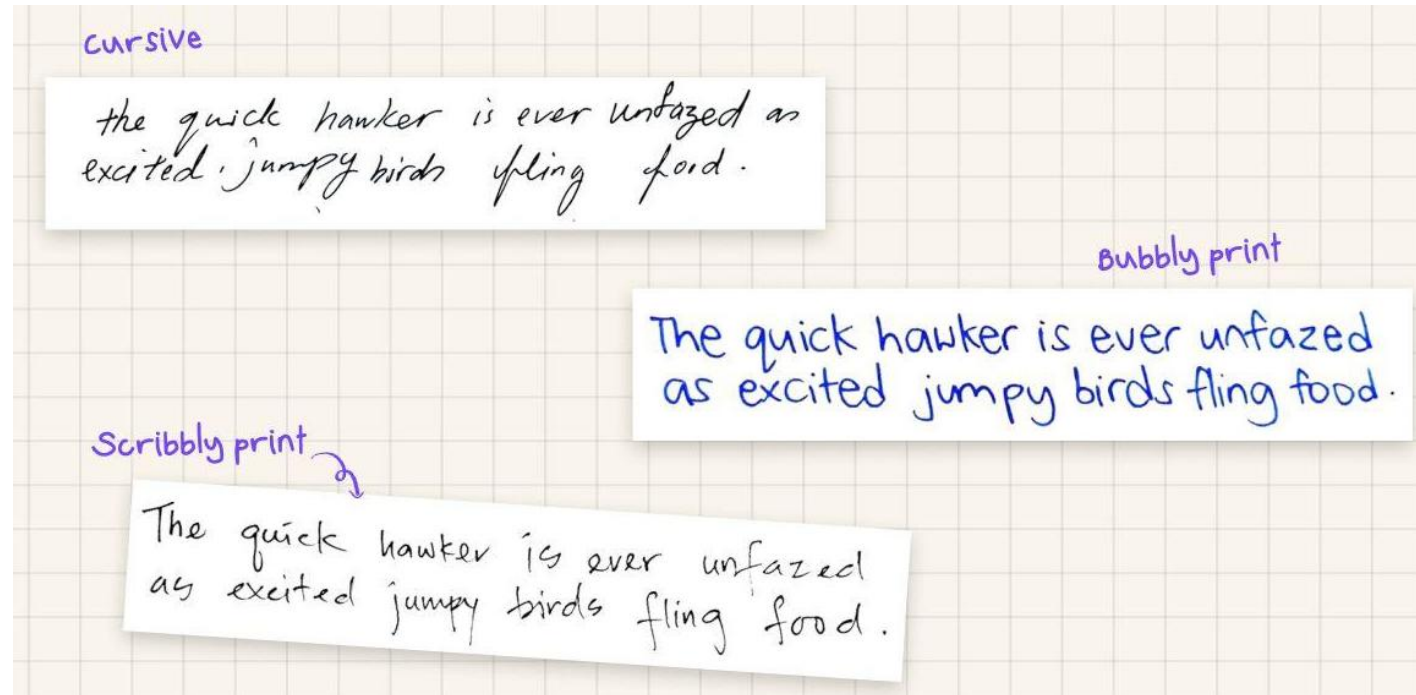
The Neocognitron introduced:

- **Shift-invariant recognition** (recognizes patterns regardless of position)
- A layered structure inspired by the **visual cortex**
- It was one of the **first models to successfully handle handwritten character recognition**

Challenges in Handwritten Character Recognition

- **Huge variation in writing styles:** Same letter/digit looks different for each person
- **Distortions and noise:** Blurred, broken, or smudged characters
- **Changes in size and orientation:** Characters may be larger, smaller, tilted, or rotated
- **Position variation:** Characters can appear anywhere in the image

👉 Traditional methods struggled because they relied on **fixed patterns** and could not adapt well to these variations.



Need for Shift-Invariant Recognition

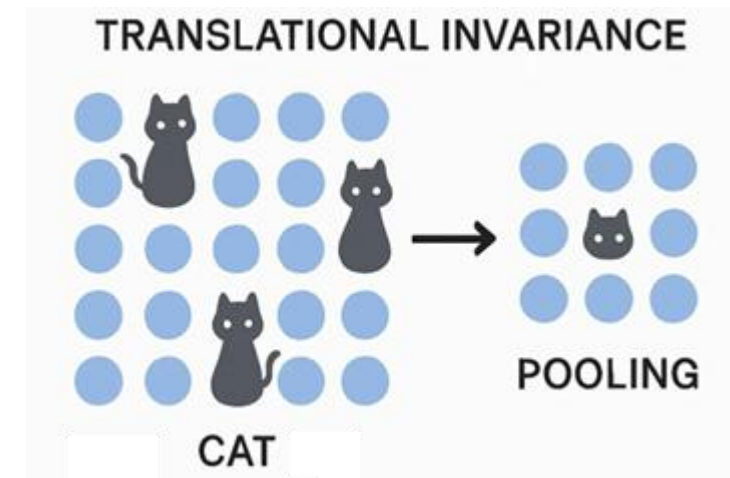
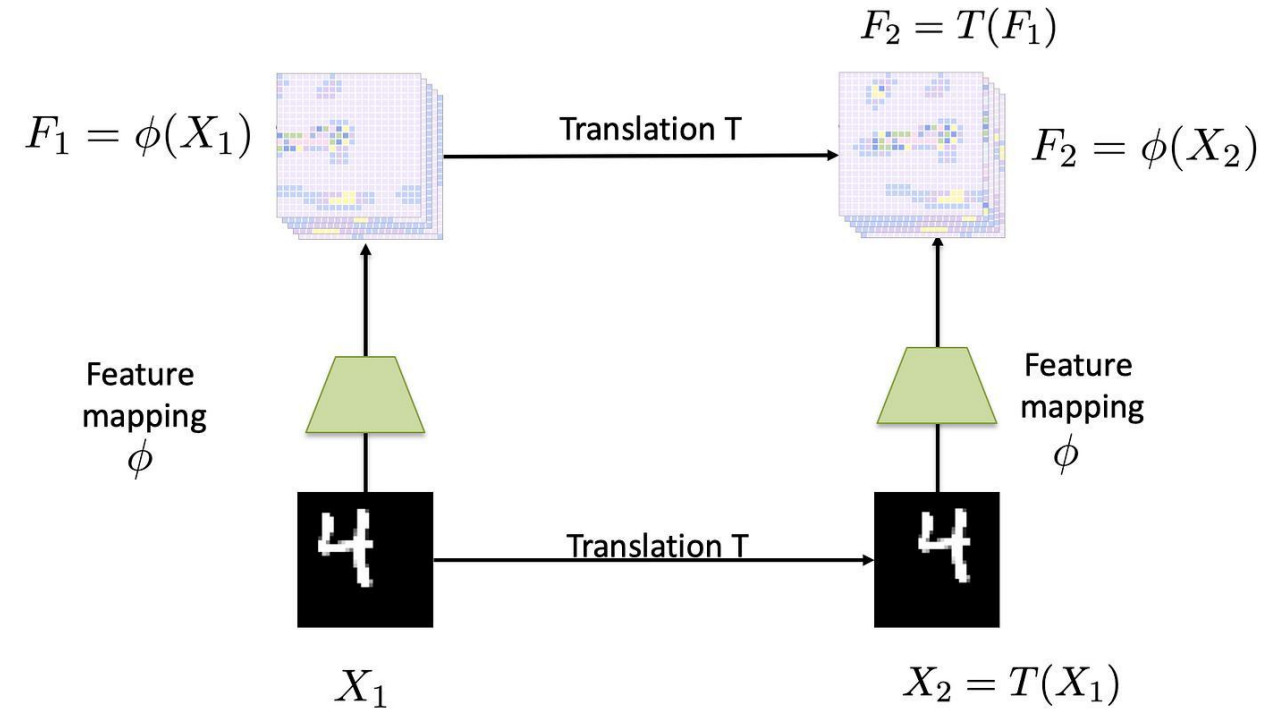
To solve these problems, a key requirement emerged:
Shift-invariant recognition (also called translation invariance)

- The system should recognize a character **no matter where it appears**
- It should handle **small shifts, distortions, and deformations**
- Inspired by how the human brain recognizes objects regardless of position

👉 Example: A handwritten “5” should be recognized as “5” whether it is: Centered / Shifted left/right / Slightly distorted

The **Neocognitron** was designed to overcome these challenges by:

- Learning **features instead of fixed patterns**
- Using layered processing to achieve **position-independent recognition**



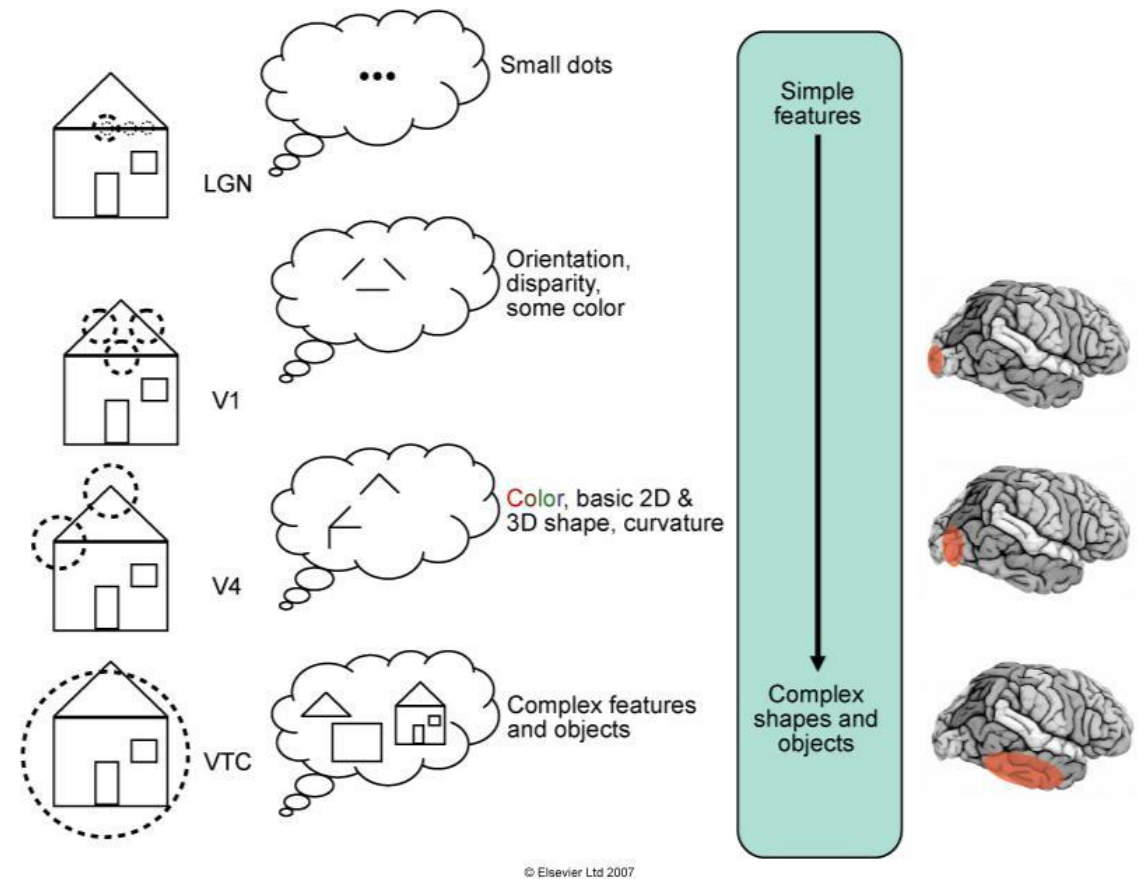
Biological Inspiration

Human Visual Cortex

- How the human brain processes visual information.
- The **visual cortex** is the part of the brain responsible for **seeing and recognizing objects layer-by-layer**
- Different neurons respond to **different visual features** like: Edges / Lines / Shapes
- Researchers David Hubel & Torsten Wiesel showed that:
 - ❑ Some cells respond to **simple patterns** (edges, orientations)
 - ❑ Others respond to **complex patterns** (shapes, objects)

It recognizes objects through **hierarchical processing**:

- Detect simple features (edges, lines)
- Combine them into patterns (corners, curves)
- Form complete objects (letters, digits)



Architecture of Neocognitron

Layered Structure Overview

The **Neocognitron** uses a **multi-layered hierarchical structure** inspired by the brain.

- Processes data **layer by layer**
- Each layer extracts **increasingly complex features**
- Consists of alternating:
 - ❑ **S-layers (Simple cells)**
 - ❑ **C-layers (Complex cells)**

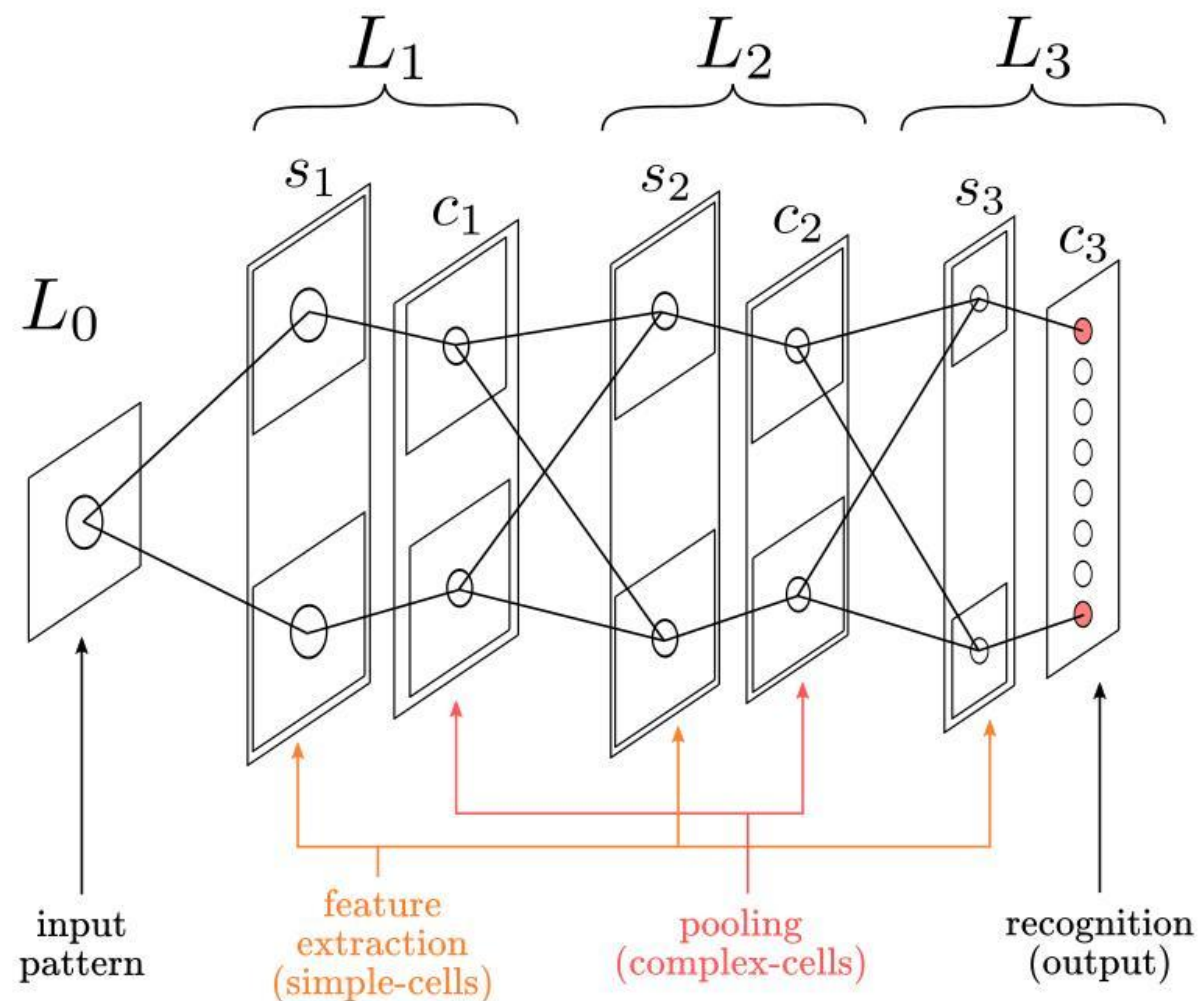
Flow:

Input → S-layer → C-layer → S-layer → C-layer → Output

S-Layers (Simple Cells)

- S-layers are responsible for **feature extraction**.
- Detect **basic features**:
 - ❑ Edges
 - ❑ Lines
 - ❑ Corners
- Similar to filters in modern CNNs
- Each neuron responds to a **specific pattern**

👉 Example: Detecting horizontal or vertical strokes in handwritten digits



Architecture of Neocognitron

C-Layers (Complex Cells)

- Provide **invariance and feature integration**
 - Combine outputs from S-layers
 - Make the system **robust to small shifts and distortions**
 - Reduce sensitivity to exact position (translation invariance)
- 👉 Similar to **pooling layers** in modern networks

Input Layer

Receives the raw image (e.g., handwritten digit)

Output Layer

Produces the **recognized character/class**

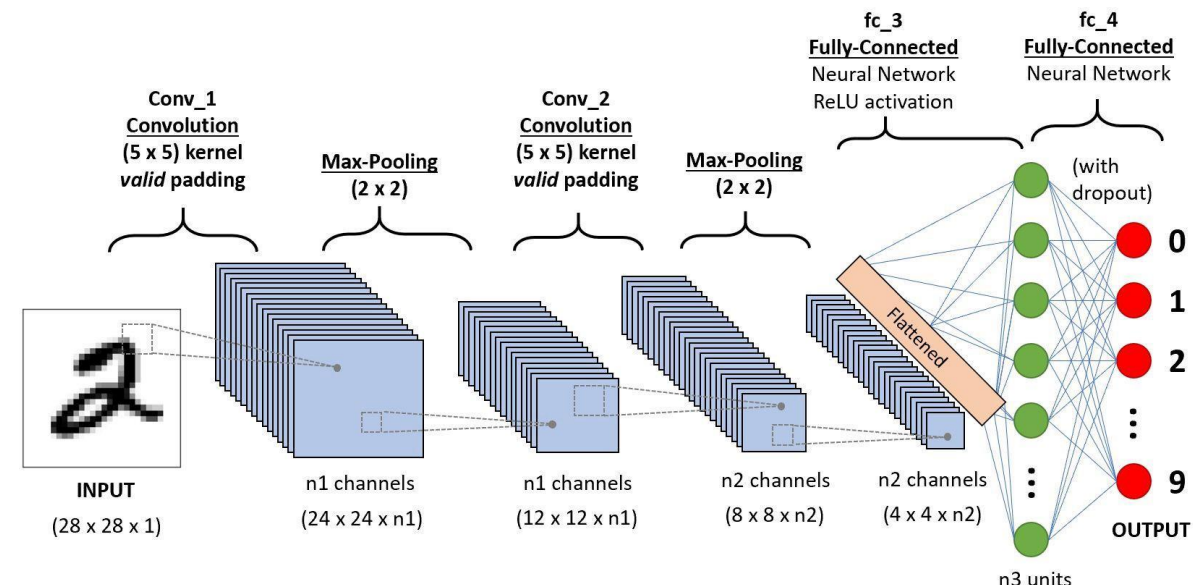
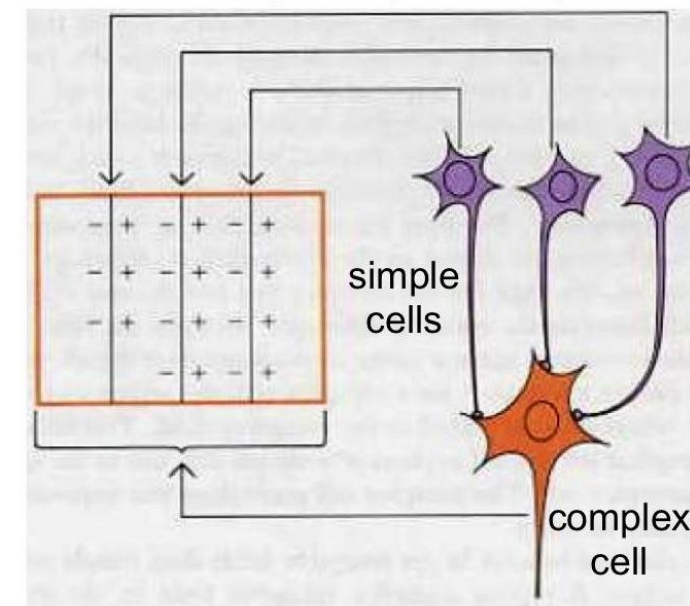
Example: Output = "5"

Architecture follows:

Simple feature detection → **Complex feature building**
→ **Final recognition**

This design became the foundation for
Convolutional Neural Networks

response of a complex cell, constructed
from multiple simple cell inputs



Working Principle of Neocognitron

1. Feature Extraction

The first step is extracting important features from the input image.

2. Pattern Matching

Next, the system matches extracted features with learned patterns.

Compares features with **stored representations**

Identifies which pattern is the **closest match**

Uses both S-layers and C-layers

👉 Example: Detected curves are matched to patterns of digits like 3, 8, or 9

3. Hierarchical Learning Process

The **Neocognitron** learns in a hierarchical way:

- Lower layers → detect simple features (edges, lines)
- Middle layers → combine into shapes (curves, parts of digits)
- Higher layers → recognize full patterns (complete digits)

4. Unsupervised Learning Approach

The **Neocognitron** uses an **unsupervised learning** approach.

- Does **not require labeled data** (no need to tell the model “this is 5”)
- Learns by **observing patterns** in input data
- Automatically adjusts connections based on:
 - ❑ Frequency of patterns
 - ❑ Similarities in features

👉 This makes it different from many modern systems that rely heavily on labeled datasets.

5. Self-Organization Concept

- A key idea is **self-organization**.
- The network **organizes itself** during learning
- Neurons become specialized to detect **specific features**
- Frequently seen patterns strengthen certain connections
- Similar to how the brain learns from experience

👉 Example: Some neurons learn to detect **horizontal lines**, Others learn **curves or loops**

Advantages & Limitations of Neocognitron

Advantages of Neocognitron

Robust to Noise and Distortion

Can recognize characters even if they are **unclear or noisy**

Handles:

- Blurred images
- Broken or incomplete strokes
- Irregular handwriting

Translation Invariance

Recognizes patterns **regardless of position**

A character is identified correctly even if it is:

- Shifted left/right/up/down

Early Foundation of CNNs

Introduced ideas used in **Convolutional Neural Networks**

Key contributions:

- Layered architecture
- Feature extraction
- Invariance handling

Limitations of Neocognitron

Computational Complexity

Requires **high computation power**

Slower, especially on older systems

Not efficient compared to modern models

Limited Compared to Modern Models

Lower performance than modern deep learning models

Lacks:

- Advanced optimization techniques
- Large-scale training capability

Applications of Neocognitron



Handwritten Digit Recognition

Used to recognize digits (0–9) from handwritten input

Example uses:

- Postal code reading

- Bank cheque processing



Optical Character Recognition (OCR)

Forms the basis of **Optical Character Recognition** systems

Converts handwritten or printed text into digital text

Widely used in:

- Document scanning

- Text digitization



Image Processing Tasks

Helps in identifying patterns in images

Used in:

- Object recognition

- Shape detection

- Basic computer vision tasks

Difference between Neocognitron vs CNNs

Aspect	Neocognitron	Convolutional Neural Networks
Historical Context	Introduced in 1980 as one of the first biologically inspired models for visual pattern recognition	Evolved from earlier models (like Neocognitron); gained major success after 2012 with deep learning advancements
Primary Goal	To achieve robust recognition of patterns (especially handwritten characters) with position invariance	To solve complex real-world problems like image classification, detection, segmentation, speech, and more
Developer / Contributors	Developed solely by Kunihiro Fukushima	Developed and improved by many researchers, including Yann LeCun
Learning Approach	Purely unsupervised learning; no labeled data required	Primarily supervised learning; also supports unsupervised and self-supervised methods
Training Mechanism	Self-organizing learning: neurons adapt based on input patterns without explicit error correction	Backpropagation: weights updated using error gradients between predicted and actual output
Architecture Structure	Fixed hierarchical structure with alternating S-layers (feature extraction) and C-layers (invariance)	Flexible and customizable architecture with convolutional, pooling, fully connected, normalization, and residual layers
Feature Learning Depth	Limited to basic and intermediate features (edges, simple shapes)	Capable of learning highly abstract features (objects, faces, complex patterns) across many layers
Handling of Invariance	Built-in translation invariance through C-layers	Achieved using pooling layers, strides, padding, and data augmentation techniques
Network Depth	Shallow network with limited number of layers	Deep architectures (can exceed 100+ layers, e.g., ResNet)

Difference between Neocognitron vs CNNs

Aspect	Neocognitron	Convolutional Neural Networks
Data Handling Capability	Works well on small datasets	Requires large-scale datasets (e.g., ImageNet) for high performance
Accuracy & Performance	Moderate accuracy; suitable for simple tasks	Very high accuracy; state-of-the-art in many AI benchmarks
Computational Requirements	High for its time; inefficient on early hardware	Optimized for modern hardware (GPUs/TPUs), enabling fast and efficient computation
Scalability	Limited scalability due to rigid design and lack of optimization	Highly scalable; can handle massive datasets and complex models
Flexibility & Adaptability	Low flexibility; architecture is mostly fixed	Highly flexible; many architectures (AlexNet, VGG, ResNet, EfficientNet, etc.)
Generalization Ability	Limited ability to generalize to highly complex or unseen data	Strong generalization when trained properly with diverse data
Real-World Applications	Handwritten character recognition, basic image processing tasks	Wide range: image recognition, object detection, NLP, speech recognition, medical imaging, autonomous driving
Current Usage	Rarely used today; mainly for academic and historical understanding	Widely used in industry, research, and real-world AI systems
Impact on AI Development	Laid the foundation for modern deep learning and CNN architectures	Forms the backbone of modern artificial intelligence and computer vision systems

NETtalk: Converting English Text to Speech

NETtalk: Converting English Text to Speech

NETtalk is one of the earliest neural network systems (1987) that learned how to **read text aloud**

Developed by:

- ❑ Terry Sejnowski
- ❑ Charles Rosenberg

Simple idea:

- It teaches a computer to read
- This is like a child learning to read English
- At first, child doesn't know pronunciation rules
- Child learn by **practice and correction**

NETtalk works the same way—it learns from examples

It converts:

Text (letters) → Speech (sounds/phonemes)

👉 Example: “cat” → /k/ /æ/ /t/

Uses a **neural network** (like a simplified brain)
Instead of hardcoding rules, it **learns patterns automatically**

👉 Key idea: English pronunciation is **not straightforward**

Real-life analogy:

“c” in *cat* sounds different from “c” in *city*

- ❑ “c” in **cat** → /k/
- ❑ “c” in **city** → /s/

Just like humans, NETtalk learns this by seeing many examples

Input Representation (Context Window)

- NETtalk does not look at one letter alone
- It looks at **neighboring letters (context)**

In real-life also when we read:

We don't read letters one by one

We look at the **whole word or nearby letters**

NETtalk does the same

NETtalk: Converting English Text to Speech

👉 **Word: cat / cake**

We want to find the sound of the letter “a”

If we look at only one letter:

“a” → could be:

/æ/ (cat)

/eɪ/ (cake)

👉 So just “a” is confusing

Using Context Window (look at neighbors)

Word: cat Window (size = 3 for simplicity):

c a t

Middle letter = **a** and Neighbors = **c and t**

👉 Now pronunciation becomes clear: “a” in **cat** → /æ/

Word: cake Window (size = 4 for simplicity):

c a k e

Same letter “a”

But neighbors are different

👉 So sound changes: “a” in **cake** → /eɪ/

Same letter + different neighbors = different sound

Simple Math Representation (Input Encoding)

Each letter is converted into numbers through **One-Hot**

Encoding. Each letter is represented as a **binary vector**

Only **one position = 1**, all others = 0, That’s why “**one-hot**”.

Simple Example

We have only 3 letters:

a, b, c

Then:

a → [1, 0, 0]

b → [0, 1, 0]

c → [0, 0, 1]

English Alphabet

Total letters = 26

Each letter → vector of size **26**

👉 Example:

a → [1,0,0,0,0,...]

b → [0,1,0,0,0,...]

z → [0,0,0,...,1]

Input Vector Formation

If context window = 7 letters

👉 Total input size: $26 \times 7 = 182$

So: $x = [x_1, x_2, \dots, x_{182}]$

Real-Life Analogy

Like giving each letter a **unique seat number**

Only one seat is occupied (1), others empty (0)

NETtalk: Converting English Text to Speech

Network Structure

Input Layer → Hidden Layer → Output Layer

👉 Mathematical operation: $y = f(Wx + b)$

Where:

x = input (letters as numbers)

W = weights (importance of connections)

b = bias (adjustment)

y = output (sound)

Real-life analogy:

Like a student combining knowledge (input) + experience (weights)

To give an answer (output pronunciation)

Output Representation

Output = **phoneme (basic sound unit)**

👉 Example: “cat” → /k/ /æ/ /t/

Real-life analogy:

Like breaking speech into **sound pieces**

Similar to how we learn spelling with sounds

Training Process

Uses **supervised learning** with

Input: text

Target: correct pronunciation

👉 Error calculation: $Error = (Target - Output)^2$

Real-life analogy:

Teacher checks student reading:

If wrong → corrects it

If right → reinforces learning

Weight Update (Learning)

The network improves using **gradient descent**

$$W_{new} = W_{old} - \eta \frac{\partial E}{\partial W}$$

Where: η = learning rate

Real-life analogy:

Like adjusting your mistake:

If you mispronounce a word

Next time, you correct it slightly

NETtalk: Converting English Text to Speech

Weight Update Example (Step-by-Step)

$$W_{new} = W_{old} - \eta \frac{\partial E}{\partial W}$$

Let's assume:

$W_{old} = 0.5$, Learning rate $\eta = 0.1$, Gradient $\frac{\partial E}{\partial W} = 0.8$

Step 1: Multiply learning rate and gradient

$$0.1 \times 0.8 = 0.08$$

Step 2: Update weight

$$W_{new} = 0.5 - 0.08$$

$$W_{new} = 0.42$$

Final Answer 🙌 New weight = **0.42**

What happened here?

Old weight was 0.5, error suggested we **reduce** it,
So, weight decreased to **0.42**

If gradient is **positive** → decrease weight

If gradient is **negative** → increase weight

Real-Life Analogy

You answered a question wrong,

Teacher says: “You went too far”

So next time, you **adjust slightly backward**

Advantages

Learns automatically (no rule coding)

Handles complex English pronunciation

One of the first successful language-learning neural networks

Limitations

Needs training data

Not as powerful as modern systems

Limited vocabulary and accuracy

NETtalk proved: Computers can **learn language like humans**

Using:

Data (examples)

Maths (learning rules)

Neural networks (brain-like models)

Recognition of Consonant–Vowel (CV) Segments

NETtalk: Converting English Text to Speech

CV segments are basic speech units formed by:

Consonant (C) → /b/, /k/, /t/

Vowel (V) → /a/, /e/, /i/

👉 Examples: “ba”, “ka”, “ti”, “do”

Speech is not continuous for understanding—we break it into **small sound blocks**

Like breaking a long sentence into **syllables** while learning to read

Example: “ba-by”, “ta-ble”

Objective: Identify and classify **CV sound units** from speech that helps machines:

- ☐ Understand speech
- ☐ Recognize words
- ☐ Process language

Real-life analogy:

Like listening carefully and identifying:

“Did the person say *ba* or *da*?”

Speech Signal Representation

Speech is a **continuous waveform (sound signal)**

It changes over time

👉 Mathematical representation:

$$s(t) = A\sin(2\pi ft)$$

Where:

A = loudness

f = pitch

Real-life analogy:

Like sound waves you see in a music app

Loud voice → bigger waves

Soft voice → smaller waves

Feature Extraction

To understand speech, we extract important features.

Key features:

Energy → how strong the sound is

Frequency → pitch of the sound

Formants → important patterns for vowels

NETtalk: Converting English Text to Speech

Energy Calculation

$$E = \sum_{n=1}^N x_n^2$$

👉 Interpretation:

Vowels → **high energy (louder, smoother)**

Consonants → **low or irregular energy**

Real-life analogy:

Vowels are like **open sounds (aaa, ooo)** → strong voice

Consonants are like **short bursts (t, k, p)** → weaker or sharp sounds

Classification Using Neural Network

Features are given to a neural network

It decides which CV segment it is

👉 Mathematical model:

$$y = f(Wx + b)$$

Where:

x = features

y = predicted CV class

Segmentation Process

Speech is divided into **small frames (chunks)**

👉 Steps:

- Break speech into small parts
- Analyze each part
- Detect consonant and vowel
- Combine → CV segment

Real-life analogy:

Like watching a slow-motion video

You analyze **frame by frame** to understand motion

Probability-Based Decision

Model calculates probability for each CV class

👉 Formula:

$$P(CV \mid x) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

👉 Choose the highest probability

Real-life analogy:

Like guessing a word based on sound:

“It is most likely *ba*, not *da*”

NETtalk: Converting English Text to Speech

1. Speech Signal

$$s(t) = A \sin(2\pi f t)$$

👉 Let's take: $A = 2$ (loudness), $f = 1$ Hz
 $t = 0.25$

Step-by-step:

$$\begin{aligned} s(0.25) &= 2 \sin(2\pi \cdot 1 \cdot 0.25) \\ &= 2 \times 1 = 2 \end{aligned}$$

👉 Output = **2 i.e.** At that moment, sound amplitude = 2

Analogy: Like measuring how loud your voice is at a specific time

2. Energy Calculation Example

$$E = \sum_{n=1}^N x_n^2$$

👉 Suppose signal samples:

$$x = [1, 2, 3]$$

Step-by-step:

$$E = 1^2 + 2^2 + 3^2 = 14$$

👉 Energy = **14 i.e.** higher value → stronger sound (likely vowel), like checking how powerful a sound is. If there is loud shout → high energy

3. Neural Network Output Example

$$y = f(Wx + b)$$

👉 Let:

$$\begin{aligned} x &= [1, 2] \\ W &= [0.5, 0.5] \\ b &= 1 \end{aligned}$$

Step-by-step:

$$\begin{aligned} Wx &= (0.5 \times 1) + (0.5 \times 2) \\ &= 0.5 + 1 = 1.5 \\ y &= f(1.5 + 1) = f(2.5) \end{aligned}$$

👉 If $f(x) = x$ (simple case):

$$y = 2.5$$

👉 Output = **2.5 i.e.** Network produces a value used to classify CV sound

Like your brain giving a score: "This sounds like 'ba' with confidence 2.5"

NETtalk: Converting English Text to Speech

4. Probability Calculation Example

$$P(CV | x) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

👉 Suppose outputs: $z_1 = 1$) ba) and $z_2 = 2$) da)

Step-by-step:

$$e^1 = 2.718, e^2 = 7.389$$
$$\text{Total} = 2.718 + 7.389 = 10.107$$

$$P(ba) = \frac{2.718}{10.107} \approx 0.27$$

$$P(da) = \frac{7.389}{10.107} \approx 0.73$$

👉 Result: “da” has higher probability, **final decision:** → **da**

Putting It All Together

Let's say input sound = “ba”

- Speech signal captured → waveform
- Energy calculated → high → likely vowel present
- Features extracted → pattern formed
- Neural network computes output

Probabilities calculated: ba = 0.8 and da = 0.2

Final output → “**ba**”

Final Understanding

Each formula plays a role:

Signal → represents sound

Energy → identifies sound strength

Neural equation → processes features

Probability → makes final decision

👉 Together, they help machine understand speech like humans

Challenges

- Background noise
- Different accents
- Fast speaking speed
- Overlapping sounds

Applications

- ☐ Speech recognition systems
- ☐ Voice assistants
- ☐ Language learning apps
- ☐ Pronunciation correction

Texture classification and segmentation

Texture classification and segmentation

Texture = pattern or appearance of a surface in an image

Examples:

Brick wall → repetitive pattern

Grass → irregular pattern

Metal → smooth

Texture helps a computer understand **what kind of surface it is seeing**

You can tell the difference between:

Sand vs water vs cloth

Even without touching—just by looking

Objective

Texture Classification → Identify *what* the texture is

Texture Segmentation → Identify *where* each texture is in the image

Classification → “This is grass”

Segmentation → “This part is grass, this part is road”

Image Representation

Image is stored as pixel values:

$I(x, y)$

👉 Example image (3×3):

$$\begin{bmatrix} 10 & 20 & 30 \\ 20 & 30 & 40 \\ 30 & 40 & 50 \end{bmatrix}$$

Meaning: Each number = brightness at that pixel

Feature Extraction

(a) Mean (Average Intensity)

$$\mu = \frac{1}{N} \sum_{i=1}^N x_i$$

👉 Values:

10, 20, 30, 20, 30, 40, 30, 40, 50

Step-by-step:

$$\begin{aligned} \mu &= \frac{10 + 20 + 30 + 20 + 30 + 40 + 30 + 40 + 50}{9} \\ &= \frac{270}{9} = 30 \end{aligned}$$

👉 Mean = **30 i.e.** average brightness of image

Texture classification and segmentation

(b) Variance (Texture Roughness)

$$\sigma^2 = \frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2$$

👉 Using mean = 30

Step-by-step:

$$(10 - 30)^2 = 400$$

$$(20 - 30)^2 = 100$$

$$(30 - 30)^2 = 0$$

$$(40 - 30)^2 = 100$$

$$(50 - 30)^2 = 400$$

$$\begin{aligned} \text{(Sum all)} &= 400 + 100 + 0 + 100 + 0 + 100 + 0 + 100 \\ &+ 400 = 1200 \end{aligned}$$

$$\sigma^2 = \frac{1200}{9} \approx 133.3$$

👉 Variance $\approx$ **133**

Meaning:

High variance $\rightarrow$ rough texture

Low variance $\rightarrow$ smooth texture

(c) Gradient (Edge Detection)

$$G = \sqrt{\left(\frac{\partial I}{\partial x}\right)^2 + \left(\frac{\partial I}{\partial y}\right)^2}$$

👉 Example: Let: Change in x = 10 , Change in y = 10

Step-by-step:

$$\begin{aligned} G &= \sqrt{10^2 + 10^2} \\ &= \sqrt{100 + 100} = \sqrt{200} \approx 14.14 \end{aligned}$$

👉 Gradient $\approx$ **14.14 i.e.** high value $\rightarrow$ strong edge or pattern

Classification Using Neural Network

$$y = f(Wx + b)$$

👉 Example: $x = [30, 133]$ (mean, variance)

$$W = [0.1, 0.05]$$

$$b = 1$$

Step-by-step:

$$Wx = (0.1 \times 30) + (0.05 \times 133)$$

$$= 3 + 6.65 = 9.65$$

$$y = 9.65 + 1 = 10.65$$

👉 Output = **10.65 i.e.** helps classify texture, like scoring a surface to decide if it's “grass” or “road”

Texture classification and segmentation

Segmentation Using Distance

$$d = \sqrt{\sum (x_i - y_i)^2}$$

👉 Example: Pixel A = [30, 100] and Pixel B = [35, 110]

Step-by-step:

$$\begin{aligned} d &= \sqrt{(30 - 35)^2 + (100 - 110)^2} \\ &= \sqrt{25 + 100} = \sqrt{125} \approx 11.18 \end{aligned}$$

👉 Distance $\approx$ **11.18**

Meaning:

Small distance $\rightarrow$ similar texture

Large distance $\rightarrow$ different texture

Analogy: Like checking similarity between two students' marks

Clustering (Grouping Pixels)

👉 Simple idea: Group pixels into clusters and minimize distance from cluster center

Analogy: Like grouping students by similar marks: Top students, Average and Below average

Challenges

Lighting changes

Rotation of textures

Scale differences

Mixed textures

Real-life analogy:

Same cloth looks different under:
Sunlight vs indoor light

Applications

Medical imaging (tumor detection)

Satellite images (land classification)

Industrial inspection (defects)

Agriculture (crop analysis)

Final Takeaway

Texture analysis helps identify:

What is in image (classification)

Where it is (segmentation)

Demo

TEACHABLE MACHINE

by Google Creative Lab

A fast, easy way to create ANN model – no coding required.

<https://teachablemachine.withgoogle.com/train>

QUICK, DRAW!

by Google Creative Lab

A game where a neural net tries to guess what you're drawing.

Can a neural network learn to recognize doodling?

Help teach it by adding your drawings to the world's largest doodling data set, shared publicly to help with machine learning research.

<https://quickdraw.withgoogle.com/>

Videos (AI ML GenAI - visual/audio/video capabilities)

[Live demo of GPT-4o coding assistant and desktop app \(youtube.com\)](#)

2.22 to 2.36

ChatGPT interprets time series temperature plot

[Live demo of GPT-4o vision capabilities \(youtube.com\)](#)

0.58 to 2.48

ChatGPT analyzes the uploaded image to identify any linear equations present.

It then utilizes its mathematical understanding to solve these equations step by step.

Users receive solutions or guidance based on the detected equations, enhancing their problem-solving experience.

TEACHABLE MACHINE - by Google Creative Lab

A fast, easy way to create ANN model – no coding required.

<https://teachablemachine.withgoogle.com/train>

QUICK, DRAW! - by Google Creative Lab

A game where a neural net tries to guess what you're drawing.

Can a neural network learn to recognize doodling?

Help teach it by adding your drawings to the world's largest doodling data set, shared publicly to help with machine learning research.

<https://quickdraw.withgoogle.com/>

Videos (AI ML GenAI - visual/audio/video capabilities)

[Seeing Theory](#)

Seeing Theory is an interactive educational platform from Brown University that visually teaches probability and statistics through interactive simulations and visualizations. Seeing Theory was created by Daniel Kunin while he was an undergraduate at Brown University, with support from the Royce Fellowship Program. Its primary goal is to make statistics more accessible and engaging by using interactive visualizations built with the D3.js JavaScript library. The platform is designed to supplement traditional textbooks and classroom instruction, helping students explore statistical concepts dynamically.

[Machine Learning Visualized — Machine](#)

[Learning Visualized](#) Book of Jupyter Notebooks that implement and mathematically derive machine learning algorithms from first-principles. The output of each notebook is a visualization of the machine learning algorithm throughout its training phase, ultimately converging at its optimal weights. Happy Learning! – [Gavin H](#)

[bbycroft.net](#) Brendan Bycroft, Software Engineer. A visualization and walkthrough of the LLM algorithm that backs OpenAI's ChatGPT. Explore the algorithm down to every add & multiply, seeing the whole process in action.

[Transformer Explainer: LLM Transformer Model](#)

[Visually Explained](#) – demo link

[Transformers Explained Visually: Learn How LLM](#)

[Transformer Models Work](#) - YouTube video

How is Transformer Explainer Implemented?

Transformer Explainer features a live GPT-2 (small) model running directly in the browser. This model is derived from the PyTorch implementation of GPT by Andrej Karpathy's [nanoGPT project](#) and has been converted to [ONNX Runtime](#) for seamless in-browser execution. The interface is built using JavaScript, with [Svelte](#) as a front-end framework and [D3.js](#) for creating dynamic visualizations. Numerical values are updated live following the user input.

Transformer Explainer was created by [Aeree Cho](#), [Grace C. Kim](#), [Alexander Karpekov](#), [Alec Helbling](#), [Jay Wang](#), [Seongmin Lee](#), [Benjamin Hoover](#), and [Polo Chau](#) at the Georgia Institute of Technology.

[Create an agent | Gemini Enterprise | Google Cloud Documentation](#)

[Create and manage agents using Agent Designer - Gemini Enterprise – Business Edition Help](#)

[A practical guide to building agents | OpenAI](#)

<https://mlvisualizer.org/>

[Home - Data Science Group - IIT Roorkee](#)

[ML Visualizer](#)

[Visual Blocks for ML](#)

[NN SVG](#)

AI for research

[arXiv.org](https://arxiv.org) e-Print archive

arXiv is a curated research-sharing platform that promotes scientific innovation and equitable access to scholarship worldwide. Since its founding in 1991, arXiv has been freely available and open to all.

arXiv now hosts over 2.9 million scholarly articles across eight subject areas. It received 284,486 submissions in 2025, representing a 17% year-over-year increase. Since surpassing the 2 million-article threshold in 2022, arXiv has seen nearly a 50% increase, with total submissions now just under 3 million.

The volunteer scientists who moderate the paper submissions are subject matter experts with terminal degrees in their fields and, along with the editorial team, ensure the quality of the content while remaining dedicated to rapid distribution. arXiv's emphasis on openness, collaboration, and scholarship provides the firm foundation from which arXiv serves the scientific community.

[Google Scholar](https://scholar.google.com) Labs

Google has introduced Scholar Labs, a new experimental feature for Google Scholar that uses generative AI to transform the process of answering detailed scholarly research questions. This move positions the platform to change how academics and researchers conduct literature reviews, potentially saving hours of manual searching.

Google Scholar Labs acts as an advanced AI-powered research assistant, designed to tackle complex, multifaceted queries that require looking at a subject from various angles, as outlined in Google's Blog, [The Keyword](#).

The tool operates by first analysing the user's question to identify all its key topics, specific aspects, and underlying relationships. It then executes a comprehensive search across Google Scholar's vast database for all these component parts simultaneously.



Let's stay connected, my phone
number +91-9922369797
For LinkedIn,
Scan this QR code!!

